Parallel Processing

Mst. Bizli Khatun
1912376113
Rabitry Hasda
2012176156

December 2024

What do you mean by parallel processing?What are the applications of it?

Parallel Parallel processing is an efficient form of information processing which emphasizes the exploitation of concurrent events in the computing process. Concurrency implies parallelism, simultaneity, and pipelining.

Applications of Parallel Processing

Parallel processing finds extensive applications in various domains, including:

1. **Scientific Simulations:**
   - Simulations of weather forecasting, climate modeling, and fluid dynamics.
   - Processing large-scale physics and chemistry models.
2. **Image and Video Processing:**
   - Real-time video rendering and image filtering.
   - Facial recognition and object detection systems.
3. **Data Analytics:**
   - Big Data processing using frameworks like Apache Hadoop and Spark.
   - Parallel algorithms for machine learning and AI.
4. **Computer Graphics and Gaming:**
   - Graphics rendering in real-time video games.
   - Animation processing in movies and simulations.
5. **Engineering and Scientific Computing:**
   - Solving partial differential equations for large-scale engineering problems.
   - Structural analysis, fluid simulations, and computational biology.
6. **Cryptography and Security:**
   - Parallel algorithms for encryption, decryption, and cryptanalysis.
   - Secure data transmission over networks.
7. **Network and Telecommunication Systems:**
   - Managing large-scale networks with concurrent connections.
   - Signal processing in 4G/5G and other telecommunication systems.
8. **Real-Time Systems:**
   - Autonomous vehicles and robotics for real-time decision-making.
   - Process control in manufacturing and industrial automation.

discuss the general trend in parallel processing

Parallel processing has evolved significantly to meet the increasing demand for high-speed and efficient computing. The following key trends highlight its development:

(a) **Multi-Core Processors:** Modern CPUs integrate multiple cores to execute tasks concurrently, enhancing performance and efficiency in desktop and server systems.

(b) **GPU Computing:** Graphics Processing Units (GPUs) are widely used for parallel tasks such as deep learning, scientific simulations, and image processing due to their ability to process thousands of threads simultaneously.

(c) **Distributed Systems:** Technologies like cloud computing, grid computing, and clusters enable parallel execution of large-scale tasks across geographically distributed systems.

(d) **Energy Efficiency:** Power-efficient hardware designs, such as dynamic voltage scaling (DVS) and workload distribution, address energy concerns in data centers and mobile devices.

(e) **Parallel Programming Models:** Frameworks like OpenMP, MPI (Message Passing Interface), and CUDA simplify the development of parallel programs for multi-core and GPU architectures.

(f) **AI and Machine Learning:** Parallel processing accelerates AI applications, including training and inference in deep neural networks, supported by specialized hardware like TPUs (Tensor Processing Units).

(g) **Hybrid Architectures:** Systems combining CPUs, GPUs, and accelerators (e.g., FPGAs) optimize performance for specific workloads in scientific, industrial, and AI domains.

(h) **Quantum Computing:** Emerging quantum technologies offer revolutionary parallel processing capabilities using quantum superposition for solving complex problems.

(i) **Real-Time Systems:** Real-time parallel processing supports applications such as robotics, autonomous vehicles, and IoT, where low-latency decision-making is critical.

Conclusion

Parallel processing continues to drive innovation in computing by enhancing speed, scalability, and energy efficiency. With advancements in hardware and software, and emerging technologies like quantum computing, it remains central to solving modern computational challenges across diverse fields.

Why are we developing parallel processing systems instead of sequential systems

Parallel processing systems are being developed over sequential systems due to the following reasons:

(a) **Performance Improvement:** Parallel systems divide tasks into smaller sub-tasks and execute them concurrently, significantly reducing execution time.

(b) **Limitations of Sequential Systems:** Single-processor systems have reached physical and speed limits due to power consumption, heat dissipation, and clock rate constraints.

(c) **Scalability:** Parallel systems scale effectively with multiple cores, processors, or distributed nodes, accommodating larger and more complex problems.

(d) **Handling Big Data:** Parallel systems efficiently process massive datasets in applications like AI, machine learning, and scientific simulations.

(e) **Real-Time Requirements:** Applications like robotics, autonomous systems, and IoT demand low-latency, real-time computations, which sequential systems cannot achieve.

In summary, parallel systems offer better performance, scalability, and energy efficiency, addressing the limitations of sequential systems.

"Parallel computing is the simultaneous use of multiple computing resources to solve a computational problem".Give some real-life examples. Write the main challenges for parallel computing

Definition

Parallel computing is the simultaneous use of multiple computing resources (processors, cores, or machines) to solve a computational problem by dividing it into smaller tasks that are executed concurrently.

Real-Life Examples of Parallel Computing

(a) **Weather Forecasting:**

- Large-scale weather prediction models use parallel processing to analyze massive datasets of atmospheric parameters.

(b) **Scientific Simulations:**

- Applications like molecular dynamics, climate modeling, and nuclear simulations rely on parallel computing for faster calculations.

(c) **Image and Video Processing:**

- Video encoding, image filtering, and rendering of computer graphics in movies and video games use parallel algorithms to improve speed and quality.

(d) **Machine Learning and AI:**

- Neural network training for deep learning requires parallel execution of matrix multiplications on GPUs or TPUs.

(e) **Cryptography:**

- Parallel processing accelerates encryption, decryption, and brute-force attacks on complex cryptographic systems.

(f) **Real-Time Robotics and Autonomous Vehicles:**

- Autonomous systems process sensor data in parallel to make low-latency decisions for navigation and obstacle avoidance.

(g) **Search Engines:**

- Web search engines like Google use parallel algorithms to index and search billions of web pages efficiently.

Main Challenges for Parallel Computing

Despite its benefits, parallel computing faces the following challenges:

(a) **Synchronization Overhead:**

- Coordinating tasks and ensuring proper data consistency between parallel processes introduce delays.

(b) **Load Balancing:**

- Uneven distribution of tasks among processors leads to idle resources and reduced efficiency.

(c) **Communication Overhead:**

- Data transfer between processors or nodes in distributed systems increases latency and reduces performance gains.

(d) **Scalability:**

- Achieving efficient scalability on large numbers of processors is difficult due to increasing communication and synchronization costs.

(e) **Programming Complexity:**
- Developing parallel algorithms and debugging parallel programs is significantly more complex than sequential programming.

(f) **Hardware Constraints:**
- Hardware limitations like memory bandwidth, power consumption, and interconnect speeds affect the overall performance.

Conclusion

Parallel computing is essential for solving complex problems efficiently in various real-world applications. However, challenges such as synchronization, load balancing, and programming complexity must be addressed to fully leverage its potential.

Definitions of Speedup, Efficiency, and Throughput

Speedup (S)

Speedup measures the performance improvement of a parallel system compared to a sequential system. It is defined as:

$$S = \frac{T_s}{T_p}$$

where T_s is the execution time of the sequential program, and T_p is the execution time of the parallel program.

Efficiency (E)

Efficiency indicates how effectively the computing resources are utilized in a parallel system. It is defined as:

$$E = \frac{S}{P} = \frac{T_s}{P \cdot T_p}$$

where P is the number of processors, S is the speedup, and T_p is the parallel execution time.

Throughput

Throughput represents the amount of work a system can process in a given period. For a parallel system, it is defined as:

$$\text{Throughput} = \frac{\text{Total number of tasks completed}}{\text{Total time taken}}$$

Summary

- **Speedup:** Measures how much faster a parallel system is compared to a sequential one.
- **Efficiency:** Reflects the utilization of processing power in a parallel system.
- **Throughput:** Quantifies the work processed per unit time.

Definitions of Multiprogramming, Time-sharing, and Multiprocessing

Multiprogramming

Multiprogramming is a technique that allows multiple programs to reside in memory simultaneously and execute by sharing the CPU. The operating system switches between programs to maximize CPU utilization, but only one program runs at a time.

Time-sharing

Time-sharing is a multitasking method where the CPU time is divided into small time slices and allocated to multiple users or processes. Each user feels as though they have exclusive access to the system due to rapid context switching.

Multiprocessing

Multiprocessing refers to a system with two or more processors working in parallel to execute multiple processes simultaneously. This enhances system throughput, reliability, and performance.

Basic Structure of an Array Computer

Definition

An Array Computer is a type of SIMD (Single Instruction, Multiple Data) architecture where a single control unit broadcasts instructions to multiple processing elements (PEs), allowing them to operate simultaneously on different data sets. It is optimized for tasks like matrix operations, image processing, and other highly parallel computations.

Structure of an Array Computer

The basic structure consists of the following key components:

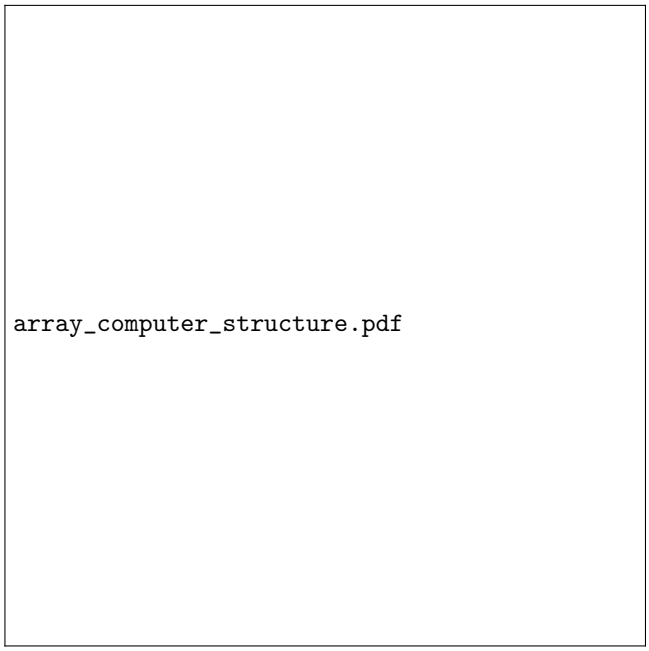
(a) **Control Unit (CU):** Responsible for fetching and broadcasting a single instruction to all PEs.

(b) **Processing Elements (PEs):** An array of identical processors that perform computations in parallel on individual data elements.

(c) **Interconnection Network:** Provides communication between PEs or between the CU and PEs.

(d) **Memory:** Each PE is associated with its own local memory for storing data.

Diagram of an Array Computer



Discussion

(a) **Control Unit:** The CU issues the same instruction to all PEs. This makes the system highly efficient for problems that can be parallelized, but it may not be ideal for tasks requiring diverse instruction sets.

(b) **Processing Elements:** The PEs operate on different data in parallel, making the array computer ideal for large-scale numerical computations like matrix multiplications, signal processing, and vector arithmetic.

(c) **Interconnection Network:** A flexible interconnection allows data exchange between PEs, enabling algorithms like sorting, data aggregation, and matrix transposition.

(d) **Memory:** Each PE has local memory to store data, reducing contention for shared memory resources.

year question 2022

1 Section A: Parallel Processing

1.1 1(a) Why develop parallel systems?

Parallel systems are developed to enhance computing performance by executing multiple tasks concurrently. The main advantages include:

- **Speedup:** Parallel systems reduce execution time for complex computations.
- **Scalability:** They can handle increasing workloads efficiently.
- **Resource Utilization:** Multi-core processors and distributed computing environments can be fully leveraged.
- **Fault Tolerance:** Some parallel architectures provide redundancy, improving system reliability.

1.2 1(b) Why write parallel programs?

Writing parallel programs allows software to take full advantage of modern multi-core and distributed architectures. Reasons include:

- **Performance Improvement:** Tasks are divided among multiple processing units.
- **Handling Large Data Sets:** Parallelism is essential in big data and scientific computations.
- **Efficiency in Resource Utilization:** CPU cores remain active rather than idle.

1.3 1(c) Exploiting Parallelism

- **Instruction-Level Parallelism (ILP):** Techniques like pipelining, out-of-order execution, and superscalar execution are used.
- **Loop-Level Parallelism:** Loop unrolling and vectorization techniques help execute multiple iterations simultaneously.
- **Task-Level Parallelism:** Different tasks are assigned to different processors for independent execution.

1.4 2(a) Flynn's Classification of Computers

Michael Flynn categorized computer architectures based on the number of instruction and data streams they process simultaneously:

- **Single Instruction, Single Data (SISD):** A traditional uniprocessor system where a single instruction operates on a single data item at a time.
- **Single Instruction, Multiple Data (SIMD):** A single instruction is executed on multiple data elements simultaneously. Example: Vector processors and GPUs.
- **Multiple Instruction, Single Data (MISD):** Multiple instructions operate on a single data stream. This architecture is rare but finds application in fault-tolerant systems.
- **Multiple Instruction, Multiple Data (MIMD):** Different processors execute different instructions on different data streams. Most modern multi-core processors and distributed systems belong to this category.

1.5 2(b) CISC vs. RISC Processor Architectures

CISC (Complex Instruction Set Computing)	**RISC (Reduced Instruction Set Computing)**
Instructions are complex and can perform multiple operations.	Instructions are simple and usually execute in one cycle.
Large instruction set with variable-length instructions.	Small instruction set with fixed-length instructions.
Requires fewer instructions but each takes more clock cycles.	Requires more instructions but each executes faster.
Microcode-based control unit.	Hardwired control unit for faster execution.
Used in older x86 processors (e.g., Intel 8086, Pentium).	Used in modern ARM, MIPS, and RISC-V architectures.

1.6 2(c) VLIW Processor Architecture

Very Long Instruction Word (VLIW) processors execute multiple independent operations in a single instruction cycle. Their key features include:

- Fixed-length instruction words containing multiple operations.
- Compiler-based instruction scheduling instead of hardware-based dynamic scheduling.

- Efficient use of available execution units.
- Examples: Intel Itanium, TI TMS320C6x.

1.7 3(a) Speedup, Efficiency, and Throughput

- **Speedup:** Measures performance improvement in parallel execution:

$$S = \frac{T_1}{T_p}, \tag{1}$$

  where T_1 is execution time on a single processor, and T_p is execution time on p processors.

- **Efficiency:** Defined as:

$$E = \frac{S}{p}. \tag{2}$$

- **Throughput:** The number of tasks completed per unit time.

1.8 3(b) Interleaved Memory Organization

Interleaved memory organization is a technique used to improve memory access speed by dividing memory into multiple banks that can be accessed in parallel.

- **High-Order Interleaving:** Consecutive addresses are assigned to the same memory bank. This results in longer access latency for sequential data access.
- **Low-Order Interleaving:** Consecutive memory addresses are spread across different memory banks, allowing parallel access and reducing latency.

This technique is commonly used in high-performance computing to optimize memory bandwidth utilization.

1.9 3(c) Interconnection Structure of a Multiprocessor System

The interconnection structure defines how multiple processors in a multiprocessor system communicate and share data. Common interconnection structures include:

- **Shared Bus:** All processors share a common communication bus. Simple but can cause contention.
- **Crossbar Switch:** Provides dedicated paths between processors and memory modules, ensuring high-speed communication.

- **Multistage Interconnection Network (MIN):** Uses multiple switching stages to connect processors and memory efficiently, reducing bottlenecks.
- **Hypercube Network:** A scalable structure where processors are connected in a multidimensional cube topology, optimizing parallel communication.

1.10 4(a) Classification of Pipeline Processors

Pipeline processors can be classified based on the levels of processing:

- **Instruction Pipeline:** Fetching, decoding, executing, and writing back are overlapped to enhance execution speed.
- **Arithmetic Pipeline:** Used for high-speed floating-point arithmetic operations, dividing operations into multiple stages.
- **Processor Pipeline:** Implements multiple instruction pipelines for superscalar execution.
- **Memory Pipeline:** Handles memory access in a pipelined manner to reduce latency.

1.11 4(b) Pipeline Floating Point Adder

A floating point adder is divided into several pipeline stages to improve performance. The general stages include:

1. **Exponent Comparison:** Aligns the exponents of the two floating-point numbers.
2. **Mantissa Addition:** Adds or subtracts the mantissas after alignment.
3. **Normalization:** Adjusts the result to fit in IEEE floating-point format.
4. **Rounding and Storage:** Rounds the result and stores it back in floating-point format.

1.12 4(c) Functional Segmentation in IBM 360/91

The IBM 360/91 utilizes a storage-to-register floating-point instruction segmentation:

- **Instruction Fetch:** Retrieves the instruction from memory.
- **Instruction Decode:** Decodes the floating-point operation.
- **Operand Fetch:** Loads operands from memory into floating-point registers.

- **Execution:** Performs arithmetic operations using floating-point functional units.
- **Result Storage:** Stores the result back to memory or registers.

This segmentation improves instruction throughput by allowing concurrent execution of different stages.

2 Section B: Database Systems

2.1 5(a) False Sharing

False sharing occurs when multiple processors modify different variables located in the same cache block, leading to unnecessary cache invalidations and performance degradation. This happens because the cache coherence protocol treats the entire block as a unit, even if different processors access different parts of it.

Example: Consider two processors modifying separate variables stored in the same cache line. When one processor updates its variable, the entire cache line is invalidated, causing unnecessary memory traffic and slowing down execution.

2.2 5(b) Mutex, Busy-Waiting, and Semaphore Differences

- **Mutex (Mutual Exclusion):** A locking mechanism that ensures only one thread can access a resource at a time. It avoids race conditions and deadlocks when used properly.
- **Busy-Waiting:** A technique where a process continuously checks for a condition to be met (e.g., spinning on a flag variable). This wastes CPU cycles and reduces efficiency.
- **Semaphore:** A synchronization tool used for controlling access to a shared resource. It can allow multiple threads to proceed (counting semaphore) or act as a binary lock (binary semaphore).

2.3 5(c) Purpose of Allgather() Function in MPI

The `Allgather()` function in Message Passing Interface (MPI) is used for collective communication where each process gathers data from all other processes and distributes the combined data to every participating process. It is commonly used in parallel computing to share computed results among all processes.

Use case: In distributed computing, if each process computes a partial result, `Allgather()` can be used to collect all results and distribute the final dataset to all nodes efficiently.

Answer to Question 6

a) Distinguish between Local and Remote Distribution

Local Distribution	**Remote Distribution**
Systems are interconnected within a limited geographical area.	Systems are interconnected over a long distance.
Uses high-speed interconnections like buses, in-plant wiring, or local area networks (LANs).	Uses telecommunications such as leased lines, satellite links, or wide area networks (WANs).
Communication latency is low due to short distances.	Communication latency is higher due to long-distance transmission.
Examples: Computer clusters, data centers, LAN-connected devices.	Examples: Cloud computing, distributed databases across multiple cities.

b) Main Factors That Trend to Distributed Processing

The factors driving the trend toward distributed processing are as follows:

- **Low cost of processors:** The decreasing cost of processors makes it feasible to use multiple smaller, cheaper processors rather than relying on a single large system.
- **Grosch's law overruled:** Grosch's law states that the cost-effectiveness of processors increases with size. However, this has been overturned, leading to the adoption of smaller, more cost-effective systems.
- **Increasing sophistication of end-users:** End-users are becoming more adept at using complex systems and applications, thus increasing the demand for distributed processing to support them.
- **End-user dissatisfaction:** End-users are often dissatisfied with the limitations of centralized systems, such as long response times, limited availability, and lack of flexibility.
- **Need for higher availability:** Distributed systems provide increased reliability and fault tolerance, ensuring continuous system availability.
- **End-user imagination:** As end-users' needs grow and their creativity evolves, distributed systems offer the flexibility to support new, innovative applications.
- **High cost of phone lines and low bandwidth:** Distributed systems allow optimization of communication, reducing the need for expensive phone lines and addressing bandwidth limitations.

- **Need for faster dialogue:** Distributed processing enables faster responses by allowing localized processing of tasks, reducing latency.
- **Excessive CPU load:** Centralized systems can become overloaded, leading to the need for distributing tasks to reduce CPU stress and improve performance.
- **Programmer shortage:** Distributed systems allow for parallel development and specialization, addressing the shortage of skilled programmers.

Horizontal vs. Vertical System Distribution

Horizontal System Distribution	**Vertical System Distribution**
Systems at the same hierarchical level are distributed across multiple nodes.	Systems are distributed across multiple hierarchical levels in a layered manner.
Focuses on improving system **scalability** and **load balancing**.	Focuses on organizing system components in a **hierarchical structure**.
Each system performs similar tasks but operates on different subsets of data or users.	Different system components specialize in distinct processing tasks.
Used in architectures like **distributed databases, web clusters, and parallel computing**.	Used in **multi-tier architectures** such as client-server or microservices-based designs.
Example: A **distributed database** where multiple servers handle different geographic regions but provide the same service.	Example: A **three-tier web application** with separate layers for the frontend, business logic, and database.

c) Basic Differences Between Function and System Distribution

Function Distribution	System Distribution
Focuses on distributing tasks and functions of the system.	Focuses on distributing system resources and infrastructure.
Distributes responsibilities based on the tasks each component will perform.	Deals with physical and logical organization of resources like storage, processing, and data.
Mainly concerned with dividing the work among various nodes or components.	Focuses on how the system as a whole is architected and how components interact.
Usually distributes functions like data processing, user interaction, or network communication.	Involves managing system-level functions, such as memory management, resource allocation, and data management.

Table 1: Difference Between Function and System Distribution

d) Different Categories of Data Distribution

The different categories of data distribution are:

- **Central data, single host:** All data is stored on a central host, and users access this data remotely.
- **Central data, multiple hosts:** The central data is distributed across multiple hosts, but there is still a central point of control.
- **Dependent hierarchical data:** Data is organized in a hierarchical manner where one level of data depends on the previous level for its structure or access.
- **Independent hierarchical data:** Data is still organized hierarchically, but each level operates independently without dependency on the other.
- **Split data:** Data is divided into parts that are stored at different locations. The pieces are later reassembled as needed.
- **Separate data:** Data is split into distinct pieces that are stored in separate locations, but these pieces may not necessarily need to be reassembled.
- **Replicated data:** Data is copied and stored in multiple locations to increase reliability and availability.
- **Heterogeneous data system:** Data may be stored in various formats and systems that differ in structure or organization, making the system heterogeneous.

- **Combinations of the above:** A system may combine several of the above data distribution models to achieve optimal performance, availability, and scalability.

Answer to Question 7

7.a)Role of Transaction Manager and Transaction Coordinator for a Distributed Database

Transaction Manager

A **Transaction Manager (TM)** is responsible for managing transactions at a local site in a distributed database system. Its roles include:

- Ensuring **ACID** (Atomicity, Consistency, Isolation, Durability) properties for local transactions.
- Maintaining **concurrency control** to prevent conflicts.
- Managing the **local transaction log** for recovery.
- Communicating with the **Transaction Coordinator** for distributed transaction execution.

Transaction Coordinator

A **Transaction Coordinator (TC)** oversees the execution of distributed transactions across multiple sites. Its roles include:

- Coordinating **two-phase commit (2PC)** or **three-phase commit (3PC)** protocols.
- Ensuring **global consistency** of transactions across sites.
- Handling **distributed deadlock detection and resolution**.
- Managing **failure recovery** in case of site crashes.

Fragmentation

Fragmentation in a distributed database refers to the process of breaking a large database into smaller, more manageable parts, called fragments, to improve performance and efficiency. These fragments are distributed across multiple sites in the network. The main goal of fragmentation is to enhance the query performance, reduce data access time, and ensure better load distribution across the network. Fragmentation can be done in different ways, based on the structure of the data.

Horizontal Fragmentation	Vertical Fragmentation
Divides the table into smaller subsets of rows (tuples), based on a certain condition (e.g., range of values).	Divides the table into smaller subsets of columns, based on attributes (e.g., grouping related columns together).
The fragmentation is based on row-wise data.	The fragmentation is based on column-wise data.
Example: Splitting the "Employees" table by department, where each fragment contains employees of a specific department.	Example: Splitting the "Employees" table by attributes, where one fragment contains employee names and another contains employee salaries.
More suitable for queries that involve filtering rows based on certain conditions.	More suitable for queries that only need specific attributes or columns.
Can improve performance when most queries involve certain ranges or sets of rows.	Can improve performance when different applications require different sets of attributes.
Helps in distributing data across multiple sites based on geographical or organizational conditions (e.g., location-based fragmentation).	Helps in optimizing storage and access to frequently accessed columns.

Table 2: Difference Between Horizontal and Vertical Fragmentation

Advantages of Horizontal Fragmentation:

- **Improved Query Performance:** Only relevant subsets of data are accessed based on the query conditions, reducing the amount of data processed.
- **Better Data Localization:** Frequently accessed data can be stored closer to the application or user, reducing access time and bandwidth usage.
- **Easier Load Balancing:** Distributing rows across different sites can help balance the load and avoid overloading any single site.
- **Reduced Network Traffic:** Queries that only involve certain rows avoid the need to transfer unnecessary data across the network.

Advantages of Vertical Fragmentation:

- **Efficient Access to Specific Columns:** Applications that need only specific columns can access just the relevant fragments, improving performance.
- **Improved Storage Efficiency:** Data that is not often used (columns) can be stored separately to save space and improve efficiency.

- **Better Security:** Sensitive columns can be stored separately, making them easier to protect with specific access controls.
- **Reduces Redundancy:** Only the necessary columns are replicated or distributed, leading to less duplication of data.

c) What Are the Main Reasons for the Failure of a Distributed Transaction?

The main reasons for the failure of a distributed transaction include:

- **Network Failure:** One of the most common reasons for transaction failure is network failure. A communication breakdown between nodes or sites can lead to transaction failure, as sites cannot communicate with each other to confirm or commit the transaction.
- **Site Failures:** A failure at any participating site in the transaction can lead to inconsistencies or incomplete transaction execution. For example, if a site crashes during the transaction, the system may be unable to complete the transaction.
- **Transaction Deadlocks:** A deadlock occurs when two or more transactions are unable to proceed because each is waiting for the other to release resources. Deadlocks can prevent transactions from making progress and lead to a failure.
- **Concurrency Issues:** Issues related to concurrency, such as conflicts between concurrent transactions, can lead to failures. For example, if two transactions try to update the same data simultaneously, inconsistencies may occur.
- **Database Locking Problems:** If a database or resource is locked by one transaction and another transaction tries to access it, the second transaction may fail due to resource unavailability or locking conflicts.
- **Two-Phase Commit (2PC) Failure:** In a distributed transaction, the 2PC protocol is often used to ensure atomicity. However, if any site fails during the 2PC protocol, the transaction may not complete successfully, leading to a failure.
- **Insufficient Resources:** If a site does not have enough resources (such as memory or disk space) to complete a transaction, the transaction may fail at that site, causing the entire distributed transaction to fail.
- **Software Bugs:** Bugs in the database software or transaction processing logic can lead to inconsistent states or incomplete transactions, causing transaction failure.
- **Security Breaches:** Unauthorized access, malicious attacks, or data corruption caused by security breaches can result in a failure of the distributed transaction, as it compromises the integrity of the system.

Answer to Question 8

a) Briefly Explain Two-Phase Commit Protocol

The **Two-Phase Commit (2PC)** protocol is a distributed algorithm used to ensure that a distributed transaction is either committed or aborted in a consistent manner, even in the presence of failures. The protocol guarantees the atomicity of the transaction, ensuring that all involved sites or nodes agree on whether the transaction should commit or abort.

Phases of Two-Phase Commit:

1. **Phase 1: Prepare Phase**
   - The coordinator sends a "prepare" message to all participants (sites).
   - Each participant logs the transaction and checks if it can commit. If it can, the participant responds with a "Yes" vote, otherwise, it sends a "No" vote.
2. **Phase 2: Commit or Abort Phase**
   - If the coordinator receives "Yes" from all participants, it sends a "Commit" message to all participants, instructing them to commit the transaction.
   - If any participant sends a "No" vote, the coordinator sends an "Abort" message, instructing all participants to abort the transaction.

Advantages: The Two-Phase Commit ensures that the transaction is atomic, meaning it is either fully committed or fully aborted, maintaining consistency in a distributed environment.

b) What is Replication? Write Some Advantages of Data Replication.

Replication is the process of duplicating data and storing it in multiple locations within a distributed database system. It is used to improve the availability, reliability, and accessibility of data by ensuring that copies of the data exist across different sites or nodes.

Advantages of Data Replication:

- **Improved Availability:** Replication ensures that if one site fails, the data is still available at another site. This provides higher system availability and resilience.
- **Fault Tolerance:** Since data is replicated across multiple sites, the system can tolerate failures without losing data or functionality, improving fault tolerance.

- **Load Balancing:** Replication allows read operations to be distributed across different replicas, reducing the load on any single site and improving performance.
- **Improved Query Performance:** Data replication can reduce latency and improve query performance, as data can be accessed from the nearest replica rather than from a distant site.
- **Backup and Recovery:** Replicated data provides an additional layer of backup, enabling faster recovery from crashes or data corruption.

c) Define Heterogeneous Distributed Databases. Write Its Advantages.

Heterogeneous Distributed Databases refer to distributed databases that consist of different types of database management systems (DBMSs) running on different platforms and hardware, which may use different data models, query languages, or storage formats. These systems are interconnected to form a distributed system where each node can communicate with others to provide a unified view of the data.

Advantages of Heterogeneous Distributed Databases:

- **Integration of Different Systems:** Heterogeneous databases allow for the integration of multiple database systems with different architectures and data models, enabling a more flexible and adaptive solution.
- **Flexibility in Choosing DBMS:** Different applications can use the best-fit database system for their needs, such as relational, object-oriented, or NoSQL databases, without requiring a single uniform system.
- **Increased Scalability:** Heterogeneous distributed databases can scale more effectively, as different DBMSs can be optimized for different workloads and used in parallel.
- **Improved Resource Utilization:** Heterogeneous systems allow better utilization of available hardware resources by leveraging the strengths of different platforms and DBMSs.
- **Reduced Dependency on Single Vendor:** Using a variety of DBMSs in a heterogeneous environment reduces the dependency on a single vendor or technology, providing greater flexibility and avoiding vendor lock-in.

Levels of Parallel Processing

1. **Job-Level Parallelism:** Independent jobs run in parallel on different processors or computers. **Example:** A cloud system running data analysis on one server while another processes customer requests.

2. **Task-Level Parallelism:** Different tasks of the same program execute simultaneously on different processors. **Example:** In a web server, one thread handles requests, another manages database queries, and another renders pages.

3. **Inter-Instruction Parallelism:** Multiple independent instructions execute in parallel within the same processor. **Example:** A superscalar processor executes an addition and a multiplication instruction at the same time.

4. **Intra-Instruction Parallelism:** A single instruction is divided into multiple parts and executed in parallel. **Example:** In vector processors, a SIMD operation adds two arrays in parallel.

Difference between Parallel Processing and Multiprocessing:

Parallel processing refers to executing multiple tasks simultaneously to improve performance, whereas multiprocessing specifically involves multiple CPUs or cores executing processes independently.

year question 2020

1(a).Why are we developing parallel systems instead of sequential systems?

Modern computing has shifted towards parallel processing due to several compelling reasons. Unlike sequential systems that process tasks one after the other, parallel systems perform multiple tasks simultaneously. This document outlines the key reasons driving this paradigm shift.

Key Reasons

1. **Performance Improvement**
   - Parallel processing allows multiple tasks to execute simultaneously, leading to faster execution compared to sequential systems.
   - As clock speed improvements in sequential processors have slowed due to physical limitations, parallelism provides an alternative to enhance performance.
2. **Scalability**
   - Parallel architectures can be scaled up by adding more processors, enabling them to handle larger and more complex computations.
   - Sequential systems have limited scalability since increasing clock speed results in more heat dissipation and power consumption.
3. **Energy Efficiency**
   - Parallel processors can distribute workloads efficiently, leading to lower power consumption compared to high-frequency sequential processors.
   - Instead of increasing a single processor's speed (which increases power usage), multiple processors can operate at lower frequencies while maintaining high throughput.
4. **Advancements in Hardware**
   - The availability of multi-core and many-core processors makes parallel computing more practical and cost-effective.
   - Technologies like GPUs, FPGAs, and specialized parallel architectures support parallel processing natively.
5. **Big Data and AI Demands**

- Modern applications, such as machine learning, real-time simulations, and big data analytics, require massive computational power, which is achievable through parallel systems.
- Sequential processing would take impractical amounts of time for these workloads.

6. **Fault Tolerance and Reliability**
   - In parallel architectures, failures in individual components can be handled without system-wide failures, improving reliability.
   - Distributed parallel systems (e.g., cloud computing) provide redundancy, ensuring continuous operation.
7. **Economic Viability**
   - Multi-core and many-core processors provide better cost-to-performance ratios compared to high-end single-core processors.
   - Cloud computing and distributed systems reduce the need for expensive supercomputers by using parallel computing techniques.

Conclusion

These factors make parallel computing the dominant paradigm in modern computing, replacing traditional sequential systems. By leveraging multiple processors simultaneously, parallel systems meet the growing demands of high-performance computing, energy efficiency, and scalability in today's data-driven world.

1(b).Why do we need to write parallel programs?

Reasons for Writing Parallel Programs

1. **Improve Performance** – Execute multiple tasks simultaneously, reducing execution time.
2. **Utilize Multi-Core Processors** – Modern CPUs and GPUs are designed for parallel execution.
3. **Enable Scalability** – Handle large datasets and complex computations efficiently.
4. **Enhance Energy Efficiency** – Reduce power consumption by distributing workloads.
5. **Support High-Performance Computing** – Essential for AI, simulations, and big data analytics.

6. **Ensure Fault Tolerance** – Improve system reliability by distributing tasks.

Parallel programming is crucial for leveraging modern hardware and solving complex problems efficiently.

1(c).How can we exploit instruction level parallelism and loop level parallelism?

1. Instruction Level Parallelism (ILP)

ILP exploits parallelism within a single processor by executing multiple instructions simultaneously. It can be achieved using:

- **Pipelining** – Overlaps execution stages of different instructions.
- **Superscalar Execution** – Executes multiple instructions per cycle using multiple execution units.
- **Out-of-Order Execution** – Dynamically reorders instructions to utilize execution units efficiently.
- **Register Renaming** – Avoids data hazards by using multiple registers for the same variable.
- **Speculative Execution** – Predicts and executes future instructions before confirmation.

2. Loop Level Parallelism (LLP)

LLP exploits parallelism by dividing loop iterations across multiple processors or cores. It can be achieved using:

- **Loop Unrolling** – Reduces loop overhead and increases parallel execution.
- **Loop Dependency Analysis** – Identifies independent iterations for parallel execution.
- **Parallel Loop Scheduling** – Assigns loop iterations dynamically or statically to processors.
- **Vectorization** – Uses SIMD (Single Instruction, Multiple Data) to process multiple data points in parallel.

Exploiting ILP and LLP enhances system performance by maximizing hardware utilization and reducing execution time.

2(a).If we fetch a cache line from main memory, where in the cache should it be placed?

When fetching a cache line from main memory, its placement depends on the cache mapping scheme:

- **Direct-mapped:** The cache line is placed in a fixed location determined by the index bits of the memory address.
- **Fully associative:** The cache line can be placed in any available cache slot, and the tag is used for lookup.
- **Set-associative:** The cache is divided into sets, and the line can be placed in any slot within the indexed set.

2(b).What is the bottleneck of Von Neumann architecture?

The Von Neumann bottleneck arises due to a single bus shared for data and instructions, limiting CPU-memory transfer speed and causing delays in execution.

2(c).Explain which function is thread safe and which is not.

In multithreading, **thread safety** refers to whether a function can be safely invoked by multiple threads simultaneously without causing data corruption or unexpected behavior.

Thread-Safe Functions

A function is considered A function is **thread-safe** if it can be safely executed by multiple threads simultaneously, either by avoiding shared data modification or using synchronization mechanisms.

Examples of Thread-Safe Functions

- **Atomic operations:** E.g., `atomic_fetch_add()` ensures uninterrupted updates.
- **Thread-local storage:** Each thread has its own data instance (`thread_local` in C++).
- **Immutable objects:** No shared state modifications.

Examples of Non-Thread-Safe Functions

- **Global data modification:** E.g., updating a shared counter without locks.
- **Non-thread-safe libraries:** E.g., certain file-handling functions.
- **Standard library functions:** E.g., `printf()` and `strtok()` with internal static state.

Ensuring Thread-Safety

- **Locks:** Mutexes prevent concurrent access.
- **Atomic operations:** Ensure uninterrupted updates.
- **Thread-local storage:** Isolates data per thread.

2(d).What are the limitations of MPI_Scatter and MPI_Gather functions?

Limitations of MPI_Scatter

- **Fixed Elements:** The number of elements must be evenly divisible by the number of processes.
- **Uniform Distribution:** Each process receives the same number of elements, limiting flexibility.
- **Memory Contiguity:** The send buffer must be contiguous, complicating non-contiguous data handling.
- **Blocking Operation:** All processes must call `MPI_Scatter`, introducing synchronization overhead.

Limitations of MPI_Gather

- **Fixed Buffer Size:** The root must allocate a buffer matching gathered data, leading to inefficiency.
- **Uneven Data Handling:** Inefficient for varying data sizes across processes.
- **Blocking Nature:** The root must wait for all processes to send data.
- **Limited Flexibility:** Assumes a static data collection pattern.
- **High Communication Overhead:** All data is sent to the root, causing potential bottlenecks.

3(a).What are the key characteristics of a vector processor?

Key Characteristics of a Vector Processor

1. **Vector Instructions:** Vector processors operate on vector data, which are arrays or sequences of data elements (e.g., numbers) processed in parallel. Instead of performing operations on a single data element, they perform the same operation on multiple elements at once using vector instructions.

2. **Multiple Data Elements:** A single instruction can process multiple data elements simultaneously, which greatly improves performance in applications requiring large-scale data processing, such as scientific computing and engineering simulations.

3. **Pipelining:** Vector processors use deep pipelining, where different stages of the vector operation (such as fetch, decode, execute, and write-back) are handled in parallel. This increases throughput and performance.

4. **Vector Registers:** They typically include vector registers, which can hold entire vectors (arrays) of data. These registers are much larger than scalar registers, allowing them to store multiple data elements for parallel processing.

5. **High Throughput:** Vector processors are designed to deliver high throughput by performing many operations in parallel. The design ensures that operations on vector data are executed quickly by utilizing multiple processing units simultaneously.

6. **SIMD (Single Instruction, Multiple Data):** Vector processors use SIMD architecture, meaning that one instruction can process multiple pieces of data at the same time. This is in contrast to traditional scalar processors that operate on a single data element per instruction.

7. **Data Parallelism:** Vector processors excel at exploiting data parallelism. This allows for simultaneous execution of operations across large datasets, significantly speeding up tasks like matrix multiplication, signal processing, and scientific simulations.

8. **Memory Bandwidth:** These processors are optimized for high memory bandwidth, as they need to fetch large blocks of data quickly to feed into the vector registers. Efficient memory access is crucial to prevent bottlenecks during computation.

9. **Vector Unit:** The vector unit is responsible for executing vector operations, such as addition, subtraction, multiplication, and division, on the vector registers. It may contain specialized hardware for floating-point operations to accelerate mathematical computations.

10. **Parallelism Granularity:** Vector processors exploit fine-grained parallelism. Each operation is broken down into smaller tasks that can be executed concurrently, which contrasts with the coarse-grained parallelism found in multi-core processors.

11. **Scalability:** Vector processors can scale well for tasks involving large datasets, making them suitable for high-performance computing environments. Their performance improves with the size of the vectors being processed.

12. **Programming Model:** Vector processors are often programmed using specialized vector programming languages or extensions to standard languages (e.g., Fortran with vector extensions) to specify vector operations efficiently.

3(b).Does false sharing generate incorrect result? Explain the effects of false sharing.

Does False Sharing Generate Incorrect Results?

No, false sharing does not cause incorrect results but leads to performance degradation due to excessive cache coherency traffic.

Effects of False Sharing

- **Increased Cache Traffic:** Cache coherency protocols trigger unnecessary synchronization between processors.
- **Performance Degradation:** Frequent cache invalidation slows down execution.
- **Wasted CPU Cycles:** Processors spend time waiting instead of computing.
- **Cache Thrashing:** Continuous cache reloading increases memory latency.
- **Unpredictable Performance:** Impact varies based on access patterns and cache organization.

Example

If two threads modify separate variables within the same cache line, caches repeatedly invalidate each other, reducing efficiency.

3(c). What is cache coherence problem? Write a program to illustrate the cache coherence problem.

Cache Coherence Problem

The **cache coherence problem** occurs in multiprocessor systems when multiple caches store copies of the same memory location. If one processor modifies a shared variable, other caches may not reflect this change, leading to inconsistencies and incorrect results in parallel programs.

Illustrative Program

```
#include <stdio.h>
#include <pthread.h>

volatile int shared_var = 0;

void* thread1(void* arg) {
    for (int i = 0; i < 1000000; ++i) shared_var++;
    return NULL;
}

void* thread2(void* arg) {
    for (int i = 0; i < 1000000; ++i) shared_var--;
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Final value: %d\n", shared_var);
    return 0;
}
```

Explanation

- Two threads increment and decrement `shared_var` concurrently.
- Due to cache coherence issues, the final result may not be zero.
- Without proper synchronization, processors may work with stale copies.

4(a).How can a receiver fing out the following values in MPI?

1. The amount of data in the message
2. THe sender of the message
3. The tag of the message

In MPI, a receiver can extract key details from a received message using the `MPI_Recv` function and the `MPI_Status` structure.

1. Amount of Data

The `MPI_Get_count` function determines the number of received elements:

```
MPI_Status status;
int count;
MPI_Recv(buffer, max_count, datatype, source, tag, comm, &status);
MPI_Get_count(&status, datatype, &count);
```

2. Sender of the Message

The sender's rank is retrieved from the `MPI_Status` structure:

```
int sender_rank = status.MPI_SOURCE;
```

3. Message Tag

The tag of the received message is obtained as:

```
int message_tag = status.MPI_TAG;
```

4(b).Why do we need MPI_reduce and MPI_Allreduce function? Show the communication pattern of MPI_Allreduce.

Why do we need MPI_Reduce and MPI_Allreduce?

`MPI_Reduce` and `MPI_Allreduce` perform reductions (e.g., sum, max) across multiple processes.

1. MPI_Reduce

Aggregates data from all processes and stores the result in a single root process.

```
MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm);
```

2. MPI_Allreduce

Similar to `MPI_Reduce`, but shares the result with all processes.

```
MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm);
```

Key Differences

- **MPI_Reduce**: Result stored only at root.
- **MPI_Allreduce**: Result available to all processes.

Communication Pattern

- Reduction occurs in a hierarchical/tree-based manner.
- `MPI_Allreduce` ensures all processes receive the final result.

4(c).Write short notes on GPU.

A GPU is a specialized circuit for accelerating image rendering and parallel computing tasks like simulations and machine learning. Unlike CPUs, GPUs have many cores optimized for parallel processing, enabling efficient handling of large datasets.

Modern GPUs, such as NVIDIA's CUDA-based models, extend beyond graphics to general-purpose computing, making them essential for high-performance applications.

5(a). Why do we use MPI_Ssend instead of MPI_Send?

`MPI_Ssend` (Synchronous send) blocks the sender until the receiver starts receiving, ensuring synchronization. In contrast, `MPI_Send` may return immediately if buffering is available, without waiting for the receiver.

5(b).A set of numbers (15, 11, 9, 16, 3, 14, 8, 7, 4, 6, 12, 10, 5, 2, 13, 1) and four processes are given. Show how processes can sort the numbers using parallel odd-even transposition sort.

The odd-even transposition sort is a parallel sorting algorithm that alternates between odd and even indexed comparisons. Given the list:

$$15, 11, 9, 16, 3, 14, 8, 7, 4, 6, 12, 10, 5, 2, 13, 1$$

and 4 processes, we perform the following steps:

Algorithm Steps

1. **Odd Phase:** Each process compares and swaps adjacent elements at odd indices. 2. **Even Phase:** Each process compares and swaps adjacent elements at even indices. 3. Repeat the above phases until the entire array is sorted.

Sorting Phases

Phase 1 (Odd Phase)

After odd phase: $11, 9, 15, 3, 14, 8, 7, 4, 6, 12, 10, 5, 2, 13, 1, 16$

Phase 2 (Even Phase)

After even phase: $9, 11, 3, 15, 8, 7, 4, 6, 12, 10, 5, 2, 13, 1, 16$

Phase 3 (Odd Phase)

After odd phase: $9, 3, 11, 8, 7, 4, 6, 10, 5, 2, 13, 1, 12, 15, 16$

Phase 4 (Even Phase)

After even phase: $3, 9, 8, 7, 4, 6, 5, 2, 10, 1, 13, 12, 15, 16$

Final Sorted Array

$1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16$

5(c). Write a program in MPI to sort a set of numbers using parallel odd even transposition sort algorithm.

The following MPI program implements the parallel odd-even transposition sort algorithm on a set of numbers.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define N 16  // Number of elements in the array

void odd_even_sort(int *arr, int rank, int size) {
    int temp;
    for (int phase = 0; phase < N; phase++) {
        if (phase % 2 == 0) {  // Even phase
            if (rank % 2 == 0 && rank < size - 1) {
                // Compare and swap with the next element
```

```
                MPI_Sendrecv(&arr[rank], 1, MPI_INT, rank + 1, 0,
                             &arr[rank + 1], 1, MPI_INT, rank + 1, 0,
                             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                if (arr[rank] > arr[rank + 1]) {
                    temp = arr[rank];
                    arr[rank] = arr[rank + 1];
                    arr[rank + 1] = temp;
                }
            }
        } else {  // Odd phase
            if (rank % 2 == 1) {
                // Compare and swap with the previous element
                MPI_Sendrecv(&arr[rank], 1, MPI_INT, rank - 1, 0,
                             &arr[rank - 1], 1, MPI_INT, rank - 1, 0,
                             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                if (arr[rank] < arr[rank - 1]) {
                    temp = arr[rank];
                    arr[rank] = arr[rank - 1];
                    arr[rank - 1] = temp;
                }
            }
        }
        // Synchronize all processes
        MPI_Barrier(MPI_COMM_WORLD);
    }
}

int main(int argc, char *argv[]) {
    int rank, size;
    int arr[N] = {15, 11, 9, 16, 3, 14, 8, 7, 4, 6, 12, 10, 5, 2, 13, 1};

    // Initialize MPI
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Scatter the array among processes
    int local_arr[N / 4];
    MPI_Scatter(arr, N / 4, MPI_INT, local_arr, N / 4, MPI_INT, 0, MPI_COMM_WORL

    // Perform the odd-even transposition sort
    odd_even_sort(local_arr, rank, size);

    // Gather the sorted subarrays back to the root process
    MPI_Gather(local_arr, N / 4, MPI_INT, arr, N / 4, MPI_INT, 0, MPI_COMM_WORLD
```

```
    // Print the sorted array in the root process
    if (rank == 0) {
        printf("Sorted array: ");
        for (int i = 0; i < N; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }

    // Finalize MPI
    MPI_Finalize();
    return 0;
}
```

Explanation of the Program:

1. The program initializes MPI and divides the array among the processes using `MPI_Scatter`.

2. The `odd_even_sort` function implements the parallel odd-even transposition sort. The function alternates between even and odd phases, where processes compare and swap elements with their neighbors.

3. After sorting, the sorted subarrays are gathered back to the root process using `MPI_Gather`.

4. The root process prints the final sorted array.

6(a).Define grain size and latency

Grain Size and Latency

Grain Size: In parallel computing, grain size refers to the amount of work done in a single parallel task or unit of computation. A finer grain size means smaller tasks with more overhead, while a coarser grain size implies larger tasks with less overhead.

Latency: Latency is the time delay between the initiation of a request and the start of the corresponding response. In parallel systems, it refers to the time taken for communication or data transfer between processors or components.

6(b).Define different types of data dependence.

In parallel computing, data dependence refers to the relationship between different instructions in terms of data flow. There are four primary types of data dependencies:

1. True Dependency (Flow Dependency): A true dependency occurs when an instruction depends on the result of a previous instruction. This is

represented as:

Instruction i must wait for instruction j if j produces data used by i.

Example:

$$\text{if } i = a + b \text{ and } j = i * c, \text{ j depends on i.}$$

2. Anti-Dependency (Write-After-Read): An anti-dependency happens when an instruction writes to a location that was read by a previous instruction. This prevents the second instruction from executing before the first instruction finishes reading the data. Example:

$$\text{if } i = a + b \text{ and } j = c + d, \text{ j overwrites the value of a or b after i reads it.}$$

3. Output Dependency (Write-After-Write): Output dependency occurs when two instructions write to the same location. The order of execution must be maintained to ensure the correct final result. Example:

$$\text{if } i = a + b \text{ and } j = a * b, \text{ both write to } a.$$

4. Control Dependency: Control dependency arises when the execution of an instruction is determined by the outcome of a conditional statement, such as an if-else branch. It is critical for instruction scheduling in parallel computing.

6(c). What are the steps of grain packing? Explain with an example.

Grain packing is a technique used in parallel computing to enhance performance by grouping small tasks (or grains) into larger units for efficient execution. The primary goal of grain packing is to reduce the overhead of parallelism by balancing the grain size of tasks to improve the utilization of processing units.

Steps of Grain Packing:

1. **Divide the Problem:** Break the problem into smaller tasks or grains. These grains represent the smallest unit of work that can be processed independently.

2. **Group the Grains:** Identify small grains and group them together to form larger units of work (bigger grains). The grouping is done to reduce communication overhead and increase efficiency.

3. **Balance the Load:** The packed grains should be balanced in size to ensure that no processor is idle while others are overloaded. Load balancing ensures that all processors work in parallel without delay.

4. **Assign Grains to Processors:** Once the grains are packed and balanced, assign the packed grains to different processors or computational units. This step ensures that the workload is efficiently distributed.

5. **Execute in Parallel:** The processors execute the assigned packed grains simultaneously. Since the grains are larger, the execution efficiency improves, and the parallel overhead is reduced.

6. **Monitor and Adjust:** After execution, monitor the performance and adjust the grain packing strategy if necessary. This step ensures that the packing method is optimal for the given task.

Example of Grain Packing:
Consider a task where a processor needs to compute the sum of an array of 100 elements, and each element requires a simple operation. Instead of assigning each element as a separate task (grain), we can group multiple elements into one grain.

- Without grain packing: Each element is assigned to a processor as an independent task. This results in high overhead due to task management and communication.

- With grain packing: We group 10 elements into one grain, so instead of 100 tasks, we now have 10 tasks. These 10 tasks are then distributed across the processors. This reduces the overhead and increases the computational efficiency.

By grouping tasks into larger grains, the communication overhead is minimized, and processors spend more time performing computations rather than handling smaller, more numerous tasks.

7(a). Can multiple threads simultaneously insert new nodes into a linked list without any problem? If not, explain how to insert the nodes and write a code segment to insert the nodes by multiple threads.

Inserting Nodes into a Linked List Using Multiple Threads

Multiple threads cannot safely insert new nodes into a linked list simultaneously without proper synchronization. The problem arises from potential race conditions when multiple threads attempt to modify the linked list structure concurrently. Specifically, if two threads simultaneously try to insert nodes, it may lead to inconsistent or corrupted list structures due to interleaved accesses to the head or other parts of the list.

To ensure safe insertion, synchronization mechanisms such as mutexes or spinlocks must be used to protect the shared resource (the linked list). Here's how we can safely insert nodes using multiple threads:

1. Use a mutex to lock the linked list before any modification (insertion).
2. Perform the insertion.
3. Release the mutex to allow other threads to access the linked list.

Code Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Define the linked list node structure
struct Node {
    int data;
    struct Node *next;
};

// Define the head of the linked list
struct Node *head = NULL;

// Mutex for synchronization
pthread_mutex_t lock;

// Function to insert a node at the beginning of the linked list
void insertNode(int value) {
    // Lock the mutex before modifying the linked list
    pthread_mutex_lock(&lock);

    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;
    head = newNode;

    // Unlock the mutex after modification
    pthread_mutex_unlock(&lock);
}

// Function to print the linked list
void printList() {
    struct Node *current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}
```

```
// Thread function to insert nodes
void* threadInsert(void* arg) {
    int value = *((int*)arg);
    insertNode(value);
    return NULL;
}

int main() {
    // Initialize the mutex
    pthread_mutex_init(&lock, NULL);

    pthread_t threads[5];
    int values[5] = {5, 10, 15, 20, 25};

    // Create threads to insert nodes
    for (int i = 0; i < 5; i++) {
        pthread_create(&threads[i], NULL, threadInsert, &values[i]);
    }

    // Wait for all threads to finish
    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    // Print the linked list
    printList();

    // Destroy the mutex
    pthread_mutex_destroy(&lock);

    return 0;
}
```

Explanation: - A mutex (‘lock‘) is used to protect the critical section where the linked list is modified. Each thread locks the mutex before inserting a new node, ensuring that only one thread can modify the list at a time. - The ‘insertNode‘ function performs the insertion at the beginning of the linked list, and each thread calls this function with a different value. - After all threads complete their execution, the linked list is printed, showing the inserted nodes.

This approach ensures that the linked list remains in a consistent state, preventing race conditions during simultaneous insertions by multiple threads.

7(b).How can we synchronize threads?

Thread Synchronization

Thread synchronization is essential to ensure that multiple threads in a program do not conflict when accessing shared resources, thus preventing issues like race conditions, deadlocks, and data inconsistencies. Several mechanisms can be used to synchronize threads in a multi-threaded environment:

1. **Mutex (Mutual Exclusion):** A mutex is a synchronization primitive used to ensure that only one thread can access a critical section of code at a time. When a thread locks a mutex, other threads must wait until the mutex is unlocked. Mutexes are commonly used to protect shared resources from concurrent access.
    - Mutexes are used for exclusive access to shared data or resources.
    - Example: `pthread_mutex_lock` and `pthread_mutex_unlock` in C.
2. **Semaphores:** A semaphore is a signaling mechanism that controls access to a shared resource by multiple threads. Semaphores can either be binary (1 or 0) or counting (allowing more than one thread to access the resource at once, up to a limit). Semaphores are used for more complex synchronization tasks such as managing a pool of resources.
    - Example: `sem_wait` and `sem_post` in POSIX systems.
3. **Condition Variables:** Condition variables are used to block a thread until a particular condition is met. These variables work in conjunction with mutexes to synchronize threads. Threads wait on a condition variable until they are notified by another thread, which could signal that a condition has changed (e.g., a resource is now available).
    - Example: `pthread_cond_wait` and `pthread_cond_signal` in C.
4. **Barriers:** A barrier is used to synchronize threads at a particular point in execution. All threads must reach the barrier before any of them can proceed. Barriers are useful in parallel algorithms where threads must work together before moving on to the next step.
    - Example: `pthread_barrier_wait` in C.
5. **Read-Write Locks:** A read-write lock allows multiple threads to read shared data simultaneously but ensures exclusive access for writing. This is useful when reading operations are frequent, and write operations are less frequent.
    - Example: `pthread_rwlock_rdlock` and `pthread_rwlock_wrlock` in C.

Conclusion: Synchronizing threads is crucial to avoid data corruption and ensure correct program execution. The choice of synchronization method depends on the specific needs of the program, such as whether exclusive or concurrent access is required.

7(c).What is mutex? Write a program using mutex.

What is a Mutex?

A **mutex** (short for mutual exclusion) is a synchronization primitive used to ensure that only one thread can access a critical section of code at a time. It prevents race conditions by allowing threads to lock and unlock shared resources. If a thread locks a mutex, other threads must wait until it releases the lock. This ensures that only one thread can modify the shared resource at any given time.

A mutex is typically used when multiple threads are trying to access or modify the same data concurrently. Without proper synchronization, this can lead to inconsistent data or unexpected behavior. Mutexes provide a simple way to coordinate thread access to shared resources.

Key points: - Mutexes are used to protect critical sections of code. - A thread must lock the mutex before accessing the critical section. - Once done, the thread must unlock the mutex to allow other threads to access the section.

Example Program Using Mutex:

Below is an example C program that demonstrates the use of a mutex to safely increment a shared counter variable by multiple threads.

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t mutex;  // Declare a mutex
int counter = 0;        // Shared resource

// Function to increment the counter
void* incrementCounter(void* arg) {
    pthread_mutex_lock(&mutex);  // Lock the mutex

    // Critical section: increment the counter
    counter++;
    printf("Counter value: %d\n", counter);

    pthread_mutex_unlock(&mutex);  // Unlock the mutex
    return NULL;
}

int main() {
```

```
    pthread_t threads[5];

    // Initialize the mutex
    pthread_mutex_init(&mutex, NULL);

    // Create 5 threads that will increment the counter
    for (int i = 0; i < 5; i++) {
        pthread_create(&threads[i], NULL, incrementCounter, NULL);
    }

    // Wait for all threads to complete
    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    // Destroy the mutex after use
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Explanation of the Program: - The program creates 5 threads, and each thread increments the shared variable `counter`. - The `pthread_mutex_lock` function is used to lock the mutex before modifying the counter, ensuring that only one thread can modify the counter at a time. - After the modification, `pthread_mutex_unlock` is called to release the mutex, allowing other threads to access the critical section. - The mutex is initialized with `pthread_mutex_init` and destroyed with `pthread_mutex_destroy`.

This ensures that the program maintains consistency of the shared `counter` variable, preventing race conditions.

8(a). What are the problems of busy-waiting?

Busy-waiting occurs when a thread repeatedly checks a condition (e.g., a flag) in a tight loop, without releasing the CPU, until the condition is met. Although it seems like a simple solution for synchronization, busy-waiting has several significant problems:

1. **Inefficient CPU Usage:** In busy-waiting, the thread continuously checks the condition without performing useful work, consuming CPU resources. This results in high CPU usage and inefficiency, especially when the thread is waiting for a long time.

2. **Starvation:** If many threads are busy-waiting, other threads may be starved for CPU time, as the waiting threads block the CPU, preventing the operating system from scheduling other tasks effectively.

3. **Increased Power Consumption:** Since the CPU remains busy in a loop, it consumes more power than it would if the thread were put to sleep or allowed to wait for an event, leading to unnecessary energy consumption, particularly in multi-core systems.

4. **Difficulty in Scalability:** In a multi-threaded system with many threads, busy-waiting does not scale well because it results in significant contention for the CPU, which negatively impacts performance as the number of threads increases.

5. **Risk of Inefficiency in Multiprocessor Systems:** In a multiprocessor system, a thread that is busy-waiting on one core might prevent other threads from using the other cores effectively, leading to poor resource utilization.

8(b).Define critical section and race condition.

Critical Section and Race Condition

Critical Section: A **critical section** refers to a part of a program or code where shared resources (such as variables, data structures, or hardware devices) are accessed or modified by multiple threads or processes. Since the shared resource is being accessed, the critical section must be carefully managed to ensure that only one thread or process can access it at a time, to avoid inconsistent or incorrect data.

Critical section management typically involves synchronization mechanisms such as mutexes, semaphores, or locks to ensure mutual exclusion and prevent concurrent access to the critical section by multiple threads.

Race Condition: A **race condition** occurs when the outcome of a program depends on the sequence or timing of uncontrollable events, such as thread execution order. In the context of a critical section, a race condition arises when multiple threads or processes attempt to access or modify shared data concurrently without proper synchronization. This can lead to inconsistent or incorrect results because the threads' actions are not coordinated.

For example, if two threads try to update the same variable at the same time, the final value of the variable may depend on the unpredictable order of execution, resulting in errors.

Conclusion: Proper synchronization techniques are needed to avoid race conditions and ensure the correct execution of programs that involve critical sections.

year question 2019

Section A - Questions and Solutions

1. (a) What do you mean by parallel processing?

Parallel processing refers to the simultaneous execution of multiple tasks or instructions to improve computational speed and efficiency.

(b) Discuss different types of data dependence.

1. *True Dependence (Read after Write - RAW)*: When an instruction depends on the result of a previous instruction. 2. *Anti-dependence (Write after Read - WAR)*: When a later instruction writes to a location before an earlier instruction reads from it. 3. *Output Dependence (Write after Write - WAW)*: When multiple instructions write to the same memory location.

(c) Fine-grain program graph for matrix multiplication with parallel scheduling.

Given the constraints (150 cycles for communication, 80 cycles for multiplication, and 15 cycles for addition), a directed acyclic graph (DAG) can be created where multiplication nodes precede addition nodes and all tasks are scheduled in parallel.

(d) What is a data-flow computer? Write a simple C program and draw its data-flow graph.

A data-flow computer executes instructions as soon as all operands become available. Unlike traditional von Neumann architecture, it doesn't follow sequential execution.

```
#include <stdio.h>
int add(int a, int b) { return a + b; }
int main() {
    int x = 2, y = 3;
    int z = add(x, y);
    printf("%d\n", z);
    return 0;
}
```

2. (a) Define speedup factor, efficiency, and throughput of a k-stage pipeline processor.

- *Speedup* $S = \frac{T_{non-pipeline}}{T_{pipeline}}$ - *Efficiency* $E = \frac{S}{k}$ - *Throughput* $= \frac{1}{T_{pipeline}}$

(b) Reservation table-based pipeline optimization.

(i) Forbidden latencies are identified by checking dependencies in the table. (ii) The state transition diagram is drawn showing execution states. (iii) Minimum average latency is computed as the inverse of throughput.

(c) Applications of vector processors.

Used in scientific computing, multimedia processing, and cryptographic applications.

3. (a) Define instruction set and discuss CISC vs RISC.

An instruction set is a collection of instructions a processor can execute. - CISC: Complex instructions, fewer lines of assembly code. - RISC: Simpler instructions, faster execution per instruction.

(b) Differences among scalar, superscalar, and vector processors.

- *Scalar*: Executes one instruction per cycle. - *Superscalar*: Executes multiple instructions per cycle. - *Vector*: Operates on entire data arrays instead of individual elements.

(c) Memory interleaving and fault tolerance.

- Low-order interleaving can tolerate faults. - High-order interleaving cannot, due to block-wise allocation.

4. (a) What is a CUDA kernel function?

A CUDA kernel function is a function executed in parallel by multiple threads on a GPU.

(b) Effects of launching CUDA kernel functions.

- Increased parallelism - Memory coalescing impacts performance - Thread divergence should be minimized

(c) CUDA kernel for vector addition.

```
__global__ void vector_add(int *A, int *B, int *C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) C[i] = A[i] + B[i];
}
```

(d) Setting grid and thread block dimensions in CUDA.

Grid and thread block dimensions are set using:
`dim3 gridDim(ceil(N/threadsPerBlock), 1, 1);`
`dim3 blockDim(threadsPerBlock, 1, 1);`

5. (a) Define bisection width and bisection bandwidth.

Bisection width is the minimum number of edges that must be removed to split a network into two equal halves. Bisection bandwidth is the total bandwidth of the edges removed during this process.

5(b) Discuss snooping cache coherence and directory-based cache coherence method.

- *Snooping Cache Coherence:* Each cache controller monitors the bus to check for memory accesses and maintains consistency by invalidating or updating its cache lines accordingly. - *Directory-based Cache Coherence:* A central directory keeps track of the state of each memory block and coordinates updates to ensure coherence among caches.

5(c) Write down Amdahl's Law.

Amdahl's Law is given by:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

where S is speedup, P is the parallelizable portion of the workload, and N is the number of processors.

6. (a) Write a program to read a vector and distribute it among processes using MPI provided function.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    int rank, size;
    int data[4] = {1, 2, 3, 4};
    int recv;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    MPI_Scatter(data, 1, MPI_INT, &recv, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received %d\n", rank, recv);

    MPI_Finalize();
    return 0;
}
```

6(b) Parallel odd-even transposition sort algorithm for sorting numbers.

- Step 1: Compare and swap adjacent elements in even indexed positions. - Step 2: Compare and swap adjacent elements in odd indexed positions. - Step 3: Repeat until sorted.

6(c) Purpose of using Allgather() function in MPI.

The `MPI_Allgather` function gathers data from all processes and distributes the collected data to all processes.

7. (a) What is false sharing? Explain with an example.

False sharing occurs when multiple processors modify different variables that reside in the same cache block, causing unnecessary invalidations.

Example: If two threads update different elements of an array stored in the same cache line, performance degrades due to frequent cache updates.

7(b) Differences between mutex, busy-waiting, and semaphore.

- *Mutex:* Provides mutual exclusion and is locked/unlocked by one thread at a time. - *Busy-waiting:* A thread continuously checks for a condition, consuming CPU cycles. - *Semaphore:* A signaling mechanism that allows multiple threads to access resources based on available permits.

7(c) Can multiple threads simultaneously delete nodes of a linked list without any problem?

No, multiple threads can cause race conditions. Solution: Use fine-grained locking or lock-free techniques.

8. (a) Desired features of a parallel language.

- Scalability, load balancing, efficient synchronization, portability, and fault tolerance.

8(b) Convert sequential code to vector code.

```
DO I = 1, N
    A(I) = B(I-1)
    B(I) = 2 * B(I)
END DO
```

Vectorized Code:

```
A(2:N) = B(1:N-1)
B(1:N) = 2 * B(1:N)
```

8(c) Optimization of vector functions.

(i) *Redundant Expression Elimination:* Removes duplicate computations to improve efficiency.

(ii) *Constant Folding:* Precomputes expressions involving constants at compile time.

year question 2018

1.b)If a fraction r of our serial program remains unparallelized, then Amdal's law says we can't get a speedup better than 1/r,Illustrate with an example.

Illustration of Amdahl's Law with an Example

Amdahl's Law provides a theoretical limit to the speedup of a program when only a portion of it is parallelized. The law is expressed as:

$$S = \frac{1}{(1-r) + \frac{r}{P}}$$

where:

- S is the speedup.
- r is the fraction of the program that is parallelizable.
- P is the number of processors.
- $(1-r)$ is the fraction of the program that remains serial.

If the fraction of the serial portion is r, Amdahl's Law implies that we cannot achieve a speedup greater than $\frac{1}{r}$.

Example: Suppose that in a program, 80% of the code is parallelizable, i.e., $r = 0.8$ and the remaining 20% is serial, i.e., $(1-r) = 0.2$. Let's calculate the maximum speedup with different numbers of processors.

For a single processor ($P = 1$):

$$S = \frac{1}{(1-0.8) + \frac{0.8}{1}} = \frac{1}{0.2 + 0.8} = 1.25$$

Thus, with only one processor, the speedup is limited to 1.25.

For 10 processors ($P = 10$):

$$S = \frac{1}{0.2 + \frac{0.8}{10}} = \frac{1}{0.2 + 0.08} = \frac{1}{0.28} \approx 3.57$$

With 10 processors, the speedup increases to 3.57.

For 100 processors ($P = 100$):

$$S = \frac{1}{0.2 + \frac{0.8}{100}} = \frac{1}{0.2 + 0.008} = \frac{1}{0.208} \approx 4.81$$

With 100 processors, the speedup reaches approximately 4.81.

Conclusion: The speedup is limited due to the serial portion of the program. As the fraction of serial code $(1-r)$ increases, the speedup tends to saturate, even as the number of processors grows. For this example, no matter how many processors are used, the speedup will never exceed $1/0.2 = 5$.

2.a)What happens when multiple process first call MPI_Send simultaneously and then call MPI_Recv?How can you make your program safe if it is unsafe due to the use of MPI_Send and MPI_Recv?

In MPI, simultaneous calls to `MPI_Send` and `MPI_Recv` by multiple processes can lead to deadlocks or race conditions if not managed properly.

Issues: 1. **Blocking Sends:** If multiple processes send messages to the same destination, they may block until `MPI_Recv` is called. 2. **Deadlocks:** If all processes call `MPI_Send` without a matching `MPI_Recv`, they may wait indefinitely.

Solutions: 1. **Non-blocking Communication:** Use `MPI_Isend` and `MPI_Irecv` to allow processes to continue execution while waiting for communication. 2. `MPI_Sendrecv`**:** Ensures a send and receive occur together, avoiding deadlocks. 3. **Synchronization:** Use `MPI_Barrier` to coordinate processes before communication.

2.b)Explain why we use MPI$_Allgather$.

MPI_Allgather

`MPI_Allgather` is a collective communication operation in MPI that gathers data from all processes in a communicator and distributes it to all processes.

Why use `MPI_Allgather`**:** 1. It allows each process to send its data to all other processes and simultaneously receive data from all other processes. 2. It simplifies code and improves efficiency by avoiding explicit point-to-point communication (e.g., `MPI_Send` and `MPI_Recv`).

Use Case: When each process has a partial result and all processes need the complete set of data for further computation, `MPI_Allgather` ensures all processes have access to the full dataset.

3.a)Suppose you have an array12,14,6,8,10,16,2,4,show all the phases of sorting the array using odd-even transposition sort algorithm.

Odd-Even Transposition Sort Algorithm

The Odd-Even Transposition Sort is a parallel sorting algorithm that works by alternating between two phases: odd-indexed and even-indexed comparisons. At each phase, the algorithm compares and swaps adjacent elements if they are out of order.

Given Array: {12, 14, 6, 8, 10, 16, 2, 4}

Step-by-Step Phases:

Phase 1: Odd Phase (Odd-indexed elements) In the odd phase, we compare elements at indices 1, 3, 5, 7 and swap if necessary.

1. Compare indices (1, 2): 12 and 14, no swap. 2. Compare indices (3, 4): 6 and 8, swap → {12, 14, 8, 6, 10, 16, 2, 4} 3. Compare indices (5, 6): 10 and 16, no swap. 4. Compare indices (7, 0): 2 and 4, swap → {12, 14, 8, 6, 10, 16, 4, 2}

Phase 2: Even Phase (Even-indexed elements) In the even phase, we compare elements at indices 0, 2, 4, 6 and swap if necessary.

1. Compare indices (0, 1): 12 and 14, no swap. 2. Compare indices (2, 3): 8 and 6, swap → {12, 14, 6, 8, 10, 16, 4, 2} 3. Compare indices (4, 5): 10 and 16, no swap. 4. Compare indices (6, 7): 4 and 2, swap → {12, 14, 6, 8, 10, 16, 2, 4}

Phase 3: Odd Phase 1. Compare indices (1, 2): 14 and 6, swap → {12, 6, 14, 8, 10, 16, 2, 4} 2. Compare indices (3, 4): 8 and 10, no swap. 3. Compare indices (5, 6): 16 and 2, swap → {12, 6, 14, 8, 10, 2, 16, 4} 4. Compare indices (7, 0): 4 and 12, swap → {4, 6, 14, 8, 10, 2, 16, 12}

Phase 4: Even Phase 1. Compare indices (0, 1): 4 and 6, no swap. 2. Compare indices (2, 3): 14 and 8, swap → {4, 6, 8, 14, 10, 2, 16, 12} 3. Compare indices (4, 5): 10 and 2, swap → {4, 6, 8, 14, 2, 10, 16, 12} 4. Compare indices (6, 7): 16 and 12, swap → {4, 6, 8, 14, 2, 10, 12, 16}

Phase 5: Odd Phase 1. Compare indices (1, 2): 6 and 8, no swap. 2. Compare indices (3, 4): 14 and 2, swap → {4, 6, 8, 2, 14, 10, 12, 16} 3. Compare indices (5, 6): 10 and 12, no swap.

Phase 6: Even Phase 1. Compare indices (0, 1): 4 and 6, no swap. 2. Compare indices (2, 3): 8 and 2, swap → {4, 6, 2, 8, 14, 10, 12, 16} 3. Compare indices (4, 5): 14 and 10, swap → {4, 6, 2, 8, 10, 14, 12, 16}

Final Sorted Array: {2, 4, 6, 8, 10, 12, 14, 16}

Conclusion: The array is now sorted in ascending order. The Odd-Even Transposition Sort continues to alternate between odd and even phases until no more swaps are required, ensuring that all elements are sorted.

3.b)Write an OpenMP program to implement odd-even transposition sort algorithm.

Below is the OpenMP program that implements the Odd-Even Transposition Sort algorithm:

Listing 1: OpenMP Program for Odd-Even Transposition Sort

```
#include <stdio.h>
#include <omp.h>

#define N 8  // Size of the array

// Function to implement Odd–Even Transposition Sort
void odd_even_transposition_sort(int arr[], int n) {
```

```
    int temp;
    int phase, i;

    // Loop for odd and even phases
    for (phase = 0; phase < n; phase++) {
        // Odd phase: Compare elements at odd indices
        #pragma omp parallel for private(temp)
        for (i = 1; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                // Swap if the elements are out of order
                temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }

        // Even phase: Compare elements at even indices
        #pragma omp parallel for private(temp)
        for (i = 0; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                // Swap if the elements are out of order
                temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
    }
}

// Function to print the array
void print_array(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    // Example array
    int arr[N] = {12, 14, 6, 8, 10, 16, 2, 4};

    printf("Original array: \n");
    print_array(arr, N);

    // Call the Odd-Even Transposition Sort
    odd_even_transposition_sort(arr, N);
```

```
    printf("Sorted array:\n");
    print_array(arr, N);

    return 0;
}
```

4.a)Discuss the efficiency of pipelined processor over non pipelined processor for a single instruction.

A non-pipelined processor executes an instruction in k clock cycles, where k is the number of stages in the instruction cycle (e.g., Fetch, Decode, Execute, Memory Access, Write-back). The total execution time for a single instruction is:

$$T_{\text{non-pipelined}} = k \times T_c$$

where T_c is the clock cycle time.

In contrast, a pipelined processor also takes k cycles for the first instruction, meaning for a **single instruction**, there is no speedup:

$$T_{\text{pipelined}} = k \times T_c$$

Efficiency in Multiple Instructions

However, when executing multiple instructions, pipelining improves throughput. If we execute n instructions:

- In a non-pipelined processor, total execution time is:

$$T_{\text{non-pipelined}} = n \times k \times T_c$$

- In a pipelined processor, the first instruction takes kT_c, but subsequent instructions complete in each cycle:

$$T_{\text{pipelined}} = (k + (n-1))T_c$$

The speedup is given by:

$$\text{Speedup} = \frac{T_{\text{non-pipelined}}}{T_{\text{pipelined}}} = \frac{n \times k \times T_c}{(k + (n-1))T_c}$$

For large n, the speedup approaches k, making pipelining significantly more efficient.

4.b)What is pipeling?To perform pipelining in a processor what steps must be followed?

Pipelining is a technique used in processors to improve instruction throughput by overlapping the execution of multiple instructions. It divides the instruction cycle into multiple stages, with each stage processing a different instruction simultaneously.

Steps to Perform Pipelining

To implement pipelining in a processor, the following steps must be followed:

1. **Instruction Breakdown:** Divide the instruction cycle into stages such as Fetch, Decode, Execute, Memory Access, and Write-back. 2. **Pipeline Register Implementation:** Introduce registers between pipeline stages to store intermediate results and prevent data loss. 3. **Clock Synchronization:** Ensure each stage operates in a synchronized manner with a uniform clock cycle. 4. **Hazard Handling:** Address data, control, and structural hazards using techniques like forwarding, branch prediction, and pipeline stalls. 5. **Instruction Scheduling:** Optimize instruction order to reduce stalls and improve efficiency.

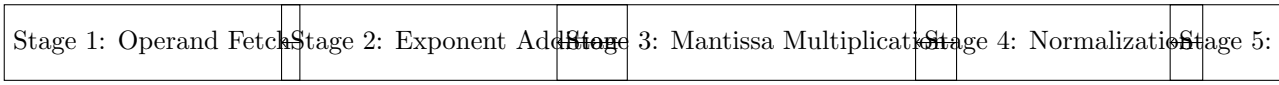
By following these steps, a processor can achieve higher instruction throughput, improving overall performance.

4.c)Draw and explain the structure of a linear pipeline for floating-point multiplication.

A linear pipeline is a sequence of processing stages where data flows in one direction, allowing multiple operations to be executed in parallel. The floating-point multiplication pipeline consists of several stages:

- **Stage 1: Operand Fetch** - Fetch the floating-point operands from registers or memory.
- **Stage 2: Exponent Addition** - Add the exponents of the two floating-point numbers.
- **Stage 3: Mantissa Multiplication** - Multiply the mantissas of the floating-point numbers.
- **Stage 4: Normalization** - Adjust the result to maintain proper floating-point format.
- **Stage 5: Rounding and Storage** - Round the result and store it in the destination register.

Pipeline Diagram



Explanation

Each stage of the pipeline processes a part of the multiplication operation. Once the first instruction moves to the next stage, a new multiplication operation enters the pipeline, improving throughput. The pipeline can achieve nearly one result per clock cycle after the initial latency, significantly enhancing floating-point multiplication performance. .

Question 5

5.a)Deifne software and hardware parallelism.

Software Parallelism: Software parallelism refers to the techniques used in software design to divide tasks into multiple independent threads or processes that can be executed concurrently. It includes approaches like task parallelism, data parallelism, and loop unrolling.

Hardware Parallelism: Hardware parallelism refers to the architectural capability of a system to execute multiple instructions or tasks simultaneously using multiple processing units. It includes features like pipelining, multiple execution units, SIMD (Single Instruction Multiple Data), and multi-core processors.

Is there any relation betwwen software and hardware parallelism.

Software and hardware parallelism are closely related as they work together to improve computational performance. Their relationship can be understood as follows:

- **Software Utilization of Hardware:** Software parallelism exploits the underlying hardware parallelism by designing programs that distribute tasks effectively across multiple processing units.
- **Hardware Dependency:** The effectiveness of software parallelism depends on hardware capabilities such as multiple cores, vector processing units, and pipelining.

- **Optimization Techniques:** Software techniques like loop unrolling, multithreading, and parallel algorithms are optimized to match hardware features like SIMD (Single Instruction Multiple Data) and MIMD (Multiple Instruction Multiple Data).
- **Scalability Considerations:** Efficient parallel software must align with hardware constraints such as memory bandwidth, cache coherence, and inter-processor communication.

Thus, software and hardware parallelism must be co-designed to achieve maximum efficiency and performance in computing systems

5(b) Relation between Software and Hardware Parallelism

Software parallelism provides tasks to be executed in parallel, while hardware parallelism provides resources for execution. An optimized program takes advantage of hardware capabilities.

5(c) Data Dependence Analysis

First set of statements:

$$S_1 : A = B + D$$
$$S_2 : C = A \times 3$$
$$S_3 : A = A + C$$
$$S_4 : E = A/2$$

Dependence Graph: $S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$

Second set of statements:

$$S_1 : X = \sin(Y)$$
$$S_2 : Z = X + W$$
$$S_3 : Y = -25 \times W$$
$$S_4 : X = \cos(Z)$$

Dependence Graph: $S_1 \rightarrow S_2 \rightarrow S_4$, S_3 (Independent)

6(a) Eliminating Communication Delays Using Node Duplication

Node duplication reduces inter-processor communication by replicating dependent data. Example: Matrix multiplication where shared elements are duplicated across processors.

6(b) Fine-Grain Program Graph for Matrix Multiplication

Given:

- Multiplication: 101 CPU cycles
- Addition: 212 CPU cycles
- Inter-processor communication: 8 CPU cycles

Perform sequential schedule (before grain packing) and parallel schedule for 4 and 6 processors.

6.a)How to elilminate communication delays between processors through node duplication?Illustrate with example.

In parallel computing, inter-processor communication can introduce significant delays. One effective technique to minimize these delays is **node duplication**, where a computation node is replicated across multiple processors to reduce data transfer overhead.

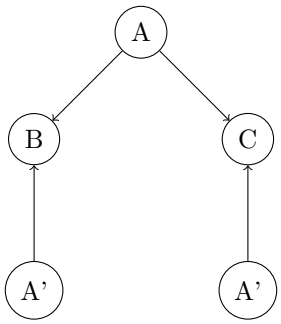
Concept of Node Duplication

Node duplication works as follows:

- A computational node that frequently exchanges data with other nodes is replicated on multiple processors.
- The replicated node allows local access to the required data, reducing inter-processor communication.
- This improves execution speed by minimizing message passing between processors.

Example:

Consider a task dependency graph where node A provides input to nodes B and C. Without duplication, data transfer is required from A to both nodes. By duplicating A, each dependent node has direct access to the required data, eliminating communication delays.



Conclusion

By applying node duplication, communication overhead is minimized, leading to improved parallel execution efficiency.

Question 7

7(a) Reservation Table Analysis

1. List forbidden latencies and collision vector.
2. Draw a state transition diagram.
3. List simple cycles from the state diagram.

7.b)What is the function of scoreboard in dynamic instructon scheduling?

The **scoreboard** is a hardware-based control mechanism used in dynamic instruction scheduling to allow out-of-order execution while avoiding hazards. Its main functions are:

- **Dependency Tracking:** It monitors instruction dependencies, including data hazards (RAW, WAR, and WAW).
- **Resource Allocation:** Ensures functional units are available before issuing instructions.
- **Hazard Resolution:** Prevents pipeline stalls by dynamically scheduling instructions to avoid conflicts.

By using a scoreboard, a processor improves parallelism and efficiency by executing independent instructions out-of-order while maintaining correctness.

7.c)How to calculate throughput of a pipoline?

Throughput is the number of instructions completed per unit time:

$$\text{Throughput} = \frac{1}{\text{Cycle Time}} \times \text{Instruction Issue Rate}$$

For an ideal k-stage pipeline:

$$\text{Throughput} = \frac{k}{\text{Total Cycles per Instruction}}$$

Question 8

8(a) Define Distributed System and Its Need

A **distributed system** is a collection of independent computers that appear to users as a single coherent system. These computers communicate and coordinate their actions via a network to achieve a common goal.

Formally, a distributed system can be defined as:

> "A system in which components located on networked computers communicate and coordinate their actions only by passing messages."

Need for Distributed Systems

The necessity of distributed systems arises due to the following factors:

- **Resource Sharing:** Enables multiple users to share hardware, software, and data efficiently.
- **Scalability:** Provides the ability to expand computing power as demand grows.
- **Fault Tolerance:** Increases reliability by distributing tasks among multiple nodes.
- **Performance Enhancement:** Parallel execution of tasks improves overall system speed.
- **Geographical Distribution:** Supports applications that require data access from different locations.
- **Cost Efficiency:** Reduces infrastructure costs by utilizing multiple low-cost machines instead of a single powerful system.

8(b) Parallel vs. Distributed Systems

Parallel Systems	**Distributed Systems**
Consist of multiple processors that work on a single problem simultaneously.	Consist of multiple independent computers that work together over a network.
Processors share memory and have tight coupling.	Nodes have independent memory and communicate via message passing.
Aimed at achieving high-speed computation through parallel execution of tasks.	Focuses on resource sharing, fault tolerance, and scalability.
Requires specialized hardware such as multi-core or multi-processor systems.	Uses general-purpose computers connected through a network.
Low communication overhead due to shared memory.	Higher communication overhead due to network-based interactions.
Example: Supercomputers like IBM Blue Gene performing scientific simulations.	Example: Cloud computing services like Google Drive or Amazon Web Services.

8(c) Naming Services in Distributed Systems

Types of Naming Services

A naming service is responsible for mapping human-readable names to system-specific identifiers such as IP addresses or object references. Some common naming services in distributed systems include:

- **Domain Name System (DNS):** Translates domain names into IP addresses.
- **Directory Services:** Used for managing and resolving names of users, devices, and services (e.g., LDAP - Lightweight Directory Access Protocol).
- **Distributed Hash Table (DHT):** A decentralized method for storing and retrieving key-value pairs (e.g., Chord, Pastry, and Kademlia).
- **Global Name Service (GNS):** Used for large-scale name resolution across a distributed system.
- **Object Naming Services:** Assigns unique names to remote objects in distributed applications (e.g., CORBA Naming Service, Java RMI Registry).

8.d)What is dstributed deadlock and why are they hard to detect?

Distributed Deadlock: Distributed deadlock occurs in a distributed system when a set of processes are each waiting for resources held by another process in the set, forming a cycle. Since the system is distributed, there is no central control to detect or resolve the deadlock.

Challenges in Detection:

- **Lack of Global Knowledge:** Each process only has local knowledge about the resources it holds or requests, making it difficult to detect cycles.
- **Asynchronous Communication:** Communication between processes is delayed, complicating the identification of dependencies and resource allocation.
- **Distributed Nature:** The absence of a central authority means no single process can observe the full system state.

year question 2017

1.b)What is 'Vector Processor'?Why do we need such processor?

Vector Processor: A vector processor is a type of CPU that processes entire vectors (arrays of data) in parallel using Single Instruction, Multiple Data (SIMD) execution. Unlike scalar processors, which handle one data element at a time, vector processors perform operations on multiple elements simultaneously, improving computational efficiency.

Need for Vector Processors: Vector processors are essential for applications requiring high-speed parallel computations, such as:

- **Scientific Computing:** Matrix operations and numerical simulations.
- **Digital Signal Processing:** Image processing, FFT, and filtering.
- **Machine Learning & AI:** Neural networks and deep learning.

Example: Vector Addition A scalar processor performs:

$$C[i] = A[i] + B[i] \quad \text{for each } i$$

whereas a vector processor executes:

$$\mathbf{C} = \mathbf{A} + \mathbf{B}$$

in a single instruction, enhancing speed and efficiency.

1.c)Can you give us a comparative study between vector and scalar processors?

Comparative Study: Vector vs. Scalar Processors

Feature	Vector Processor	Scalar Processor
Processing Type	Processes multiple data elements simultaneously	Processes one data element at a time
Instruction Execution	Single instruction operates on multiple data (SIMD)	Single instruction operates on a single data (SISD)
Speed	Faster for large data sets	Slower for large data sets
Registers	Uses vector registers to store multiple elements	Uses scalar registers for single values
Applications	Used in AI, DSP, and scientific computing	Used in general-purpose computing

Table 3: Comparison of Vector and Scalar Processors

2.a)Discuss different types of data dependence.

Types of Data Dependence:

Data dependence occurs when instructions in a program share data, leading to execution constraints. The main types of data dependence are:

1. **Flow Dependence (True Dependence)** Also known as *Read After Write (RAW)*, this occurs when an instruction depends on the result of a previous instruction.

$$A = B + C \quad \text{(Instruction 1)}$$

$$D = A + E \quad \text{(Instruction 2, depends on A)}$$

2. **Anti Dependence** Also called *Write After Read (WAR)*, this occurs when an instruction writes to a register or memory location before a previous instruction reads it.

$$A = B + C \quad \text{(Instruction 1, reads A)}$$

$$A = D + E \quad \text{(Instruction 2, writes A)}$$

3. **Output Dependence** Also called *Write After Write (WAW)*, this occurs when two instructions write to the same variable or memory location.

$$A = B + C \quad \text{(Instruction 1, writes A)}$$

$$A = D + E \quad \text{(Instruction 2, writes A again)}$$

4. **Control Dependence** This occurs when the execution of an instruction depends on the outcome of a conditional statement.

$$\text{if } (A > B) \text{ then } C = D + E$$

   The instruction modifying C is dependent on the evaluation of the condition.

2.b)What are the steps of grain packing?Discuss with an example.

Steps of Grain Packing:

Grain packing is a technique in parallel computing used to group computations into different levels of granularity for efficient execution. The steps involved are:

1. **Identifying Computation Tasks:** Break the problem into multiple tasks that can be executed in parallel.

2. **Classifying Granularity:** Determine the level of granularity:
    - **Fine-Grain:** Small tasks with frequent communication.
    - **Medium-Grain:** Balanced tasks with moderate communication.
    - **Coarse-Grain:** Large tasks with minimal communication.
3. **Grouping Tasks:** Similar tasks are grouped into larger units based on dependencies and data sharing.
4. **Mapping to Processors:** Assign grouped tasks to different processors for parallel execution.
5. **Optimizing Execution:** Minimize communication overhead and balance workload distribution.

Example: Matrix Multiplication
Consider multiplying two 4×4 matrices A and B to produce matrix C.

$$C[i][j] = \sum_{k=0}^{3} A[i][k] \times B[k][j]$$

- **Fine-Grain:** Each multiplication $A[i][k] \times B[k][j]$ is a separate task.
- **Medium-Grain:** Each row of A computes a full row of C.
- **Coarse-Grain:** The entire matrix multiplication is divided among processors.

By selecting the appropriate grain size, computational efficiency and parallel execution performance are optimized.

2.c)What is data-flow computer?Write a program and draw its dataflow graph.

Data-Flow Computer:
A **data-flow computer** is a type of parallel computing architecture where program execution is determined by the availability of data rather than a sequential instruction order. Instead of using a traditional program counter, data-flow computers execute instructions as soon as their input operands are ready.
Characteristics of Data-Flow Computers:

- Instructions are represented as a **data-flow graph**.
- Execution occurs only when all required data is available.
- High potential for parallelism as multiple instructions can be executed simultaneously.

- Avoids the need for explicit control flow mechanisms like loops and conditionals.

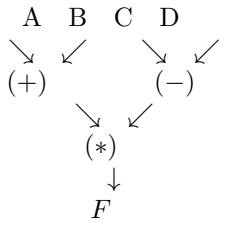
Example Program: Consider the arithmetic expression:

$$F = (A + B) \times (C - D)$$

A simple C-like code representation:

```
F = (A + B) * (C - D);
```

Data-Flow Graph:
Below is the LaTeX representation of the data-flow graph for this expression:



Explanation:

1. The operands A and B are input to an addition operation.
2. The operands C and D are input to a subtraction operation.
3. The results of these two operations are then input to a multiplication operation.
4. The final result F is produced once multiplication is completed.

This data-flow representation illustrates how operations execute as soon as their inputs are available, enabling parallel execution of independent computations.

3.a)Define Whetstone results,TPS and KLIPS ratings.

Whetstone Results, TPS, and KLIPS Ratings

- **Whetstone Benchmark:** The Whetstone benchmark is a synthetic benchmark designed to measure the floating-point and fixed-point arithmetic performance of a computer system. It evaluates the execution speed of common operations such as loops, function calls, and arithmetic operations. The performance is typically expressed in **Million Whetstone Instructions Per Second (MWIPS)**.

- **Transactions Per Second (TPS):** TPS is a performance metric used in database systems and transaction processing environments. It represents the number of transactions a system can process per second. Higher TPS values indicate better system performance, especially in high-throughput applications like banking, e-commerce, and enterprise computing.
- **Kilo Logical Inferences Per Second (KLIPS):** KLIPS is a performance metric used to measure the execution speed of logic-based systems, particularly in artificial intelligence (AI) and expert systems. It quantifies the number of logical inferences a system can process per second, commonly used in Prolog-based or rule-based AI engines.

3.b)Explain harmonic mean speedup performance for a microprocessor operating in n execution modes with repect to probalility distributions.

Harmonic Mean Speedup Performance

The performance of a microprocessor operating in n execution modes can be analyzed using the concept of **harmonic mean speedup**, which accounts for the execution time in different modes weighted by their probability of occurrence.

Definition: If a processor operates in n different execution modes, each mode i has:

- A speedup S_i relative to a baseline performance.
- A probability of occurrence P_i, where $\sum_{i=1}^{n} P_i = 1$.

The harmonic mean speedup S_{HM} is given by:

$$S_{\text{HM}} = \frac{1}{\sum_{i=1}^{n} \frac{P_i}{S_i}}$$

Explanation:

1. The harmonic mean accounts for the fact that execution speedup is inversely proportional to time.
2. It provides a better measure when execution times vary significantly across different modes.
3. Unlike the arithmetic mean, the harmonic mean is dominated by the slowest execution mode, making it a more realistic performance metric.

Example:

Consider a processor operating in three execution modes with the following parameters:

Mode i	Speedup S_i	Probability P_i
1	5	0.4
2	10	0.3
3	2	0.3

The harmonic mean speedup is computed as:

$$S_{\text{HM}} = \frac{1}{\left(\frac{0.4}{5} + \frac{0.3}{10} + \frac{0.3}{2}\right)}$$

This provides an effective performance metric for evaluating microprocessor speedup across varying execution conditions.

3.b)Define average parallelism.)

Definition of Average Parallelism:

Average parallelism P_{avg} measures the degree of parallel execution in a system. It is defined as the ratio of total work done to the critical path length of execution.

$$P_{\text{avg}} = \frac{T_{\text{total}}}{T_{\text{critical}}}$$

where:

- T_{total} = total number of instruction cycles across all parallel tasks.
- T_{critical} = execution time of the longest dependent sequence of instructions.

Explanation: Higher average parallelism indicates better utilization of parallel processing capabilities. It helps in analyzing the scalability and efficiency of parallel architectures.

4.a)What are the reasons behind the development of reduced instruction sets.

Reasons Behind the Development of Reduced Instruction Sets (RISC)

Reduced Instruction Set Computing (RISC) was developed to address inefficiencies in traditional Complex Instruction Set Computing (CISC) architectures. The key reasons behind its development are:

1. **Complexity of CISC Instructions:** CISC architectures included a large number of complex instructions, many of which were rarely used. This led to increased decoding time and inefficiencies in execution.
2. **Instruction Execution Speed:** RISC architectures focus on executing simple, frequently used instructions in a single clock cycle, improving performance and reducing instruction cycle time.

3. **Efficient Pipeline Utilization:** Simplified instructions allow for better pipeline design, minimizing hazards and increasing instruction throughput.

4. **Reduced Hardware Complexity:** Since RISC processors use fewer and simpler instructions, hardware design is simplified, leading to faster clock speeds and reduced power consumption.

5. **Optimized Compiler Performance:** RISC architectures enable compilers to generate optimized machine code, making better use of available registers and reducing memory access delays.

6. **Scalability and Energy Efficiency:** With a simpler design, RISC processors consume less power and are easier to scale, making them ideal for mobile and embedded systems.

RISC architectures prioritize simplicity, efficiency, and high-speed execution, leading to their widespread adoption in modern computing.

4.b)Draw the architecture of MC68040 microprocessor and discuss its major units.

Architecture of MC68040 Microprocessor

The Motorola MC68040 is a high-performance CISC microprocessor that integrates advanced features like an on-chip Floating-Point Unit (FPU) and Memory Management Unit (MMU). Its architecture consists of several major units:

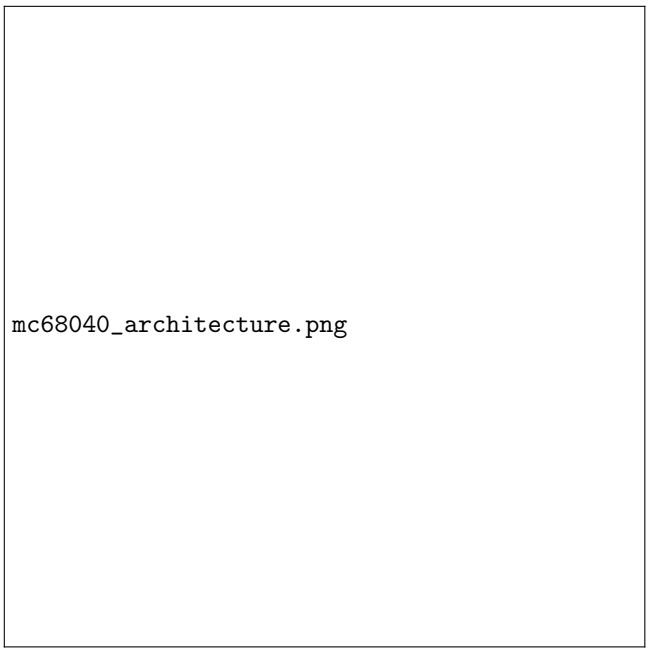


Figure 1: Architecture of MC68040 Microprocessor

Major Units of MC68040:

1. **Integer Unit (IU):** Executes arithmetic and logic operations on integer data. It includes a 32-bit Arithmetic Logic Unit (ALU) for high-speed processing.
2. **Floating-Point Unit (FPU):** Performs floating-point arithmetic operations, including addition, multiplication, division, and transcendental functions.
3. **Instruction Fetch Unit (IFU):** Prefetches instructions from memory to improve execution efficiency. It works with the instruction cache to reduce memory access delays.
4. **Instruction Cache and Data Cache:** MC68040 includes **4 KB instruction cache** and **4 KB data cache** to minimize memory latency and speed up execution.
5. **Memory Management Unit (MMU):** Supports virtual memory management and provides memory protection features, making the processor suitable for multitasking environments.
6. **Bus Interface Unit (BIU):** Handles communication between the processor and external devices, including RAM, ROM, and peripherals. It manages memory access and data transfers.

7. **Pipeline Execution:** The MC68040 features a **six-stage pipeline** for instruction execution, enabling increased parallelism and faster performance.

Conclusion: The MC68040 microprocessor integrates multiple processing units with caching and pipelining techniques, leading to improved computational efficiency and performance in various applications.

Explain the concept of using overlapped register windows introduced by the Berkeley RISC architecture.

Overlapped Register Windows in Berkeley RISC Architecture

The concept of **overlapped register windows** was introduced in the Berkeley RISC architecture to reduce the overhead associated with function calls and context switching. Instead of using a single global register file, the processor employs a set of overlapping register windows to optimize procedure calls and returns.

Key Concepts:

1. **Register Windows:** Instead of storing function arguments and local variables in memory, the processor uses multiple register sets, where each function call gets its own window of registers.
2. **Overlapping Windows:** Each register window consists of three parts:
   - **Input Registers:** Passed from the calling function.
   - **Local Registers:** Used within the function.
   - **Output Registers:** Shared with the next function, acting as its input registers.
3. **Fast Function Calls:** Since output registers of a caller automatically become the input registers of a callee, function calls and returns occur without memory access, significantly reducing execution time.
4. **Circular Register Buffer:** The architecture maintains a circular buffer of register windows. When the buffer is full, the least recently used window is saved to memory to accommodate new function calls.
5. **Efficient Context Switching:** Overlapping registers reduce register spill/fill operations during procedure calls and context switches, improving performance.

Illustration of Register Overlapping:

Function	Register Type	Shared With
Caller	Output Registers	Callee (Input Registers)
Callee	Input Registers	Caller (Output Registers)

Advantages:

- Reduces the need for memory access during function calls.
- Minimizes function call latency and improves execution speed.
- Optimizes recursive function calls by limiting memory traffic.

Conclusion: Overlapped register windows significantly enhance the efficiency of procedure calls in RISC architectures by leveraging register reuse and minimizing memory accesses, making them a key feature in early RISC processor designs.

5.a)What do you know about the distributed database.

A **distributed database** is a collection of databases that are stored across multiple physical locations but appear as a single unified system to users. These locations can be connected through a network.

Key Characteristics:

- Data is distributed across multiple sites or nodes.
- Supports distributed query processing and transaction management.
- Provides data redundancy and fault tolerance.

Example: A banking system where customer data is stored across multiple branches but accessed as a single database.

Describe the horizontal and vertical fragmentation of dristributed database.

Horizontal and Vertical Fragmentation in Distributed Databases

In a distributed database, **data fragmentation** is used to divide a database into smaller pieces, called **fragments**, which are distributed across multiple locations for improved performance, availability, and security.

1. Horizontal Fragmentation:

- In **horizontal fragmentation**, the database table is divided into subsets of rows (tuples), where each fragment contains a subset of the records based on a specific condition.
- This fragmentation is useful when different locations need access to different subsets of records.

Example: Consider a `Customers` table:

CustomerID	Name	City
101	*Alice*	*NewYork*
102	*Bob*	*LosAngeles*
103	*Charlie*	*NewYork*
104	*David*	*Chicago*

If a company has branches in different cities, a horizontal fragmentation based on `City` can create fragments:

- **Fragment 1 (New York)**: Contains rows where `City` = 'New York'. - **Fragment 2 (Los Angeles)**: Contains rows where `City` = 'Los Angeles'. - **Fragment 3 (Chicago)**: Contains rows where `City` = 'Chicago'.

2. Vertical Fragmentation:

- In **vertical fragmentation**, the database table is divided into smaller fragments based on columns (attributes).
- Each fragment contains only a subset of attributes but includes the primary key to allow reconstruction of the original table.
- This is useful when different applications require access to different attributes of the same table.

Example: Consider the same `Customers` table. A vertical fragmentation can split it into:

- **Fragment 1 (Basic Info)**:

CustomerID	Name
101	*Alice*
102	*Bob*
103	*Charlie*
104	*David*

- **Fragment 2 (Location Info)**:

CustomerID	City
101	*NewYork*
102	*LosAngeles*
103	*NewYork*
104	*Chicago*

Comparison:

- **Horizontal Fragmentation** is useful for distributing data based on access locality, reducing network overhead.
- **Vertical Fragmentation** is beneficial when different applications need different attributes, reducing data transfer.

Conclusion: Both fragmentation techniques improve query performance, reduce data redundancy, and enhance scalability in distributed databases.

Describe the failures handlings techniques in two-phase commit protocol.

Failure Handling Techniques in Two-Phase Commit Protocol

The **Two-Phase Commit (2PC)** protocol ensures that all participants in a distributed transaction either commit or abort the transaction, ensuring atomicity. Failure handling is critical for maintaining the integrity of the protocol in the presence of system crashes or communication failures. The following techniques are used to handle failures in 2PC:

1. Coordinator Failure: In the event of a failure of the coordinator (the node that initiates the commit process), the following steps are taken:

- If the coordinator fails after sending the `VOTE COMMIT` message, participants (also known as cohorts) will wait for a timeout. Upon recovery, the coordinator will either issue a `COMMIT` or `ABORT` based on the information stored in its log.
- If the coordinator fails before sending the `VOTE COMMIT`, the participants will timeout and decide to abort the transaction. When the coordinator recovers, it checks the logs of participants to determine the outcome of the transaction.

2. Participant Failure: When a participant (cohort) fails during the commit process:

- If a participant fails after sending a `VOTE COMMIT`, it will use logs to recover. Upon recovery, it will either commit or abort the transaction, depending on the coordinator's final decision.
- If a participant fails before sending a vote, the coordinator will not receive a vote from that participant and will eventually decide to `ABORT`. After recovery, the participant will learn the outcome from the coordinator and take the appropriate action.

3. Message Loss or Network Partition: If messages between the coordinator and participants are lost or a network partition occurs:

- Timeouts are used to detect such failures. If no acknowledgment is received within a specified time period, a participant or coordinator will abort the transaction.
- Upon recovery from a partition, nodes will use logs to check the transaction's status. If the coordinator's decision is unclear, the system may initiate a new protocol round (retry).

4. Logging and Recovery: The use of **persistent logs** is essential for recovery after failures. Both the coordinator and participants maintain logs of their actions:

- Logs store information about transaction state, votes, and decisions made, allowing recovery after failure.
- In case of a failure, logs allow the system to reconstruct the state of the transaction and apply the correct outcome.

Conclusion: Failure handling in the Two-Phase Commit protocol is crucial to ensure consistency in distributed systems. Techniques such as logging, timeouts, and coordination between participants ensure that transactions are either fully committed or fully aborted, even in the case of failures.

6.a)Define reversation table.

A **reservation table** is a data structure used in pipeline-based processors to manage the availability of functional units and ensure that instructions are executed in the correct order. It tracks the reservation of resources, such as functional units and registers, during instruction execution.

Purpose: The reservation table helps to prevent hazards and ensures that instructions are scheduled efficiently without conflicts, particularly in Tomasulo's algorithm used for dynamic instruction scheduling.

6.c)Explain how to reduce minimal average latency.

Reducing Minimal Average Latency

Minimal average latency refers to the time taken to execute tasks in a system, averaged over multiple operations or processes. To reduce this latency, several techniques can be employed:

1. Pipelining: Pipelining breaks down a task into smaller stages, where each stage processes a portion of the task simultaneously. This allows multiple operations to be executed concurrently, thereby reducing the overall time for processing.

2. Parallel Processing: Utilizing multiple processing units (e.g., multicore processors) to handle different tasks in parallel can significantly decrease latency. Each task or operation can be assigned to a different core, thus reducing the time for completion.

3. Data Prefetching: Prefetching data into the cache before it is needed can reduce memory access time, as waiting for data to be fetched from slower storage can be a major contributor to latency.

4. Caching: Implementing efficient caching mechanisms can reduce access times to frequently used data. By keeping frequently accessed data in a high-speed memory (e.g., CPU cache), the system avoids repeated access to slower main memory.

5. Load Balancing: Distributing tasks efficiently across available resources ensures that no single resource is overburdened, which can prevent bottlenecks and reduce overall latency.

6. Reducing Context Switching: Minimizing the overhead associated with context switching (the process of switching between tasks or threads) can reduce latency in multitasking environments. Efficiently managing task scheduling and CPU allocation can help achieve this.

7. Optimizing Algorithms: Optimizing algorithms to reduce unnecessary computations or to take advantage of specialized hardware features can also help in lowering the average latency. Efficient algorithms that minimize time complexity lead to faster execution.

Conclusion: By using pipelining, parallel processing, data prefetching, caching, load balancing, reducing context switching, and optimizing algorithms, minimal average latency can be significantly reduced, leading to more efficient systems.

7.a)What are the challenges in the development of future general-purpose supercomputers.

Challenges in the Development of Future General-Purpose Supercomputers

The development of future supercomputers faces several key challenges:

- **Power Consumption and Heat Dissipation:** Supercomputers require vast amounts of power, and as processing power increases, so does the heat generated. Efficient cooling and power management are critical.
- **Scalability:** As the number of processors grows, efficient interconnects and data communication between processors become more complex, posing scalability issues.
- **Memory Bottleneck:** With increasing processing speed, memory bandwidth and low latency become a limitation, requiring better memory subsystems.
- **Parallelism and Software Optimization:** Supercomputers rely on parallel computing, but designing software that can efficiently run on thousands or millions of cores remains a challenge.

Addressing these challenges is essential to building more powerful, energy-efficient, and scalable supercomputers.

7.b)Discuss different types of vector instructions.

Types of Vector Instructions

Vector instructions are used in vector processors to handle operations on large sets of data in parallel. They allow multiple data elements to be processed with a single instruction, improving performance in applications like scientific computing and signal processing. The primary types of vector instructions are:

1. Arithmetic Instructions: These instructions perform arithmetic operations on entire vectors or elements of vectors. The operations include addition, subtraction, multiplication, and division. Examples are:

- `VADD`: Adds corresponding elements of two vectors.
- `VSUB`: Subtracts corresponding elements of two vectors.
- `VMUL`: Multiplies corresponding elements of two vectors.
- `VDIV`: Divides corresponding elements of two vectors.

2. Data Movement Instructions: Data movement instructions are used to load and store vectors to and from memory or move vector elements between registers. These instructions are essential for transferring large data sets efficiently. Examples include:

- `VLOAD`: Loads a vector from memory into a vector register.
- `VSTORE`: Stores a vector from a register to memory.
- `VSHIFT`: Shifts the contents of a vector to the left or right.

3. Comparison Instructions: Comparison instructions allow for element-wise comparisons between two vectors, producing a result based on specified conditions. These conditions can include equality, greater than, or less than comparisons. Examples include:

- `VCMP`: Compares corresponding elements of two vectors.
- `VCEQ`: Compares for equality.
- `VCLT`: Compares for less than.

4. Reduction Instructions: Reduction instructions reduce a vector into a single scalar value by applying an operation across all the elements of the vector. Examples include:

- `VREDUCE`: Reduces the elements of a vector using a specified operation, such as summing the elements.
- `VADDRED`: Performs addition reduction, summing all the elements of a vector.

5. Control Flow Instructions: Control flow vector instructions modify the flow of execution based on the result of vector conditions. Examples include:

- `VIF`: Conditional vector instruction that executes based on a vector condition.
- `VJUMP`: Transfers control to a new location if a vector condition is true.

6. Scatter and Gather Instructions: Scatter and gather instructions are used to access non-contiguous memory locations, improving efficiency when dealing with sparse data. Examples include:

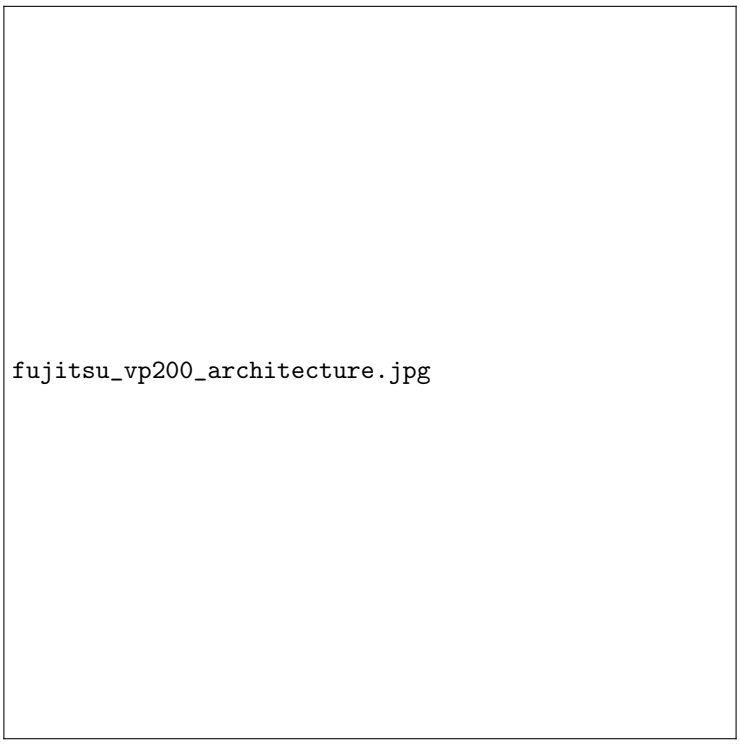
- `VSCATTER`: Scatters vector elements to multiple locations in memory.
- `VGATHER`: Gathers elements from various memory locations into a vector register.

Conclusion: Vector instructions enable parallel processing of large data sets by applying operations on entire vectors at once. This makes them critical for high-performance computing tasks, enabling faster computation in fields like scientific simulations, data analysis, and graphics rendering.

7.c)Draw the architecture of Fujitsu VP200 series supercomputer and discuss the major units.

Fujitsu VP200 Series Supercomputer Architecture

The Fujitsu VP200 series is a vector processing supercomputer designed for high-performance tasks, such as scientific computing and simulations. It incorporates several key architectural components that enable high throughput for vector operations. The architecture of the VP200 consists of multiple processing units, memory systems, and interconnection networks, optimized for parallel vector computation.



Architecture Diagram of Fujitsu VP200 Series (example)

Major Units of the Fujitsu VP200 Architecture:

1. Vector Processor Units: The heart of the Fujitsu VP200 is the vector processor, which performs operations on large vectors of data. Each vector processor includes specialized functional units optimized for high-speed arithmetic operations on vector data, such as addition, multiplication, and division.

2. Scalar Processor Unit: The scalar processor handles non-vector (scalar) operations and manages control flow in the system. While vector processors handle the bulk of computational work, the scalar processor ensures that instructions and data are provided to the vector processors efficiently.

3. Memory Units: The VP200 series uses a hierarchical memory system to support high throughput. This typically consists of:

- **Local Memory:** Each vector processor is paired with its own local memory, providing fast access to data for the vector processing unit.
- **Shared Memory:** There is also a shared memory pool, allowing multiple processors to access data concurrently. This memory is used for communication between processors and for storing larger datasets.

4. Interconnection Network: The interconnection network in the VP200 is responsible for communication between the different processing units and memory units. The network is designed to minimize communication bottlenecks,

providing high-speed data transfer between vector processors and memory. The use of a high-bandwidth interconnection network is crucial for maintaining performance in a parallel computing system.

5. I/O Units: Input and output units manage the transfer of data between the supercomputer and external systems, such as storage devices, other computers, and peripheral hardware. The I/O system in the VP200 supports high-speed data transfers to accommodate the large amounts of data generated by scientific simulations and computations.

6. Control Unit: The control unit coordinates the operations of the entire supercomputer. It manages the execution of instructions, controls data flow between the processors and memory, and handles synchronization tasks required in a parallel processing environment.

7. Vector Pipeline: The vector pipeline is a specialized processing path designed for vector operations. It allows for continuous processing of vector elements, improving throughput by overlapping computation and memory access.

Conclusion: The Fujitsu VP200 series supercomputer is designed for high-performance computing tasks, with a focus on vector processing and parallelism. Its architecture includes multiple vector and scalar processors, hierarchical memory, and high-speed interconnects, making it well-suited for large-scale scientific simulations and data-intensive computations.

8.a

Reducing Execution Time Using Different Modes

Given the pure scalar loop:

```
Do k = 1, 2048A(k) = A(k) + SEnddo
```

This loop can be optimized in several ways to reduce execution time. The optimization depends on how the operations are executed in parallel using different processing modes.

1. Vector Mode: In vector mode, the loop operates on entire vectors of data rather than individual scalar elements. The vector processor can perform the operation on multiple elements of the array A simultaneously, drastically reducing execution time. Instead of updating each $A(k)$ sequentially, the vector processor processes all $A(k)$ values in parallel, completing the operation in a fraction of the time.

`A = A + S` (This processes the entire vector of A in one instruction)

2. Scalar-Concurrent Mode: In scalar-concurrent mode, multiple scalar processors are used to perform operations concurrently. Each processor is responsible for updating a specific element of $A(k)$. The loop is still scalar in nature, but different iterations are handled by different processors. This parallelism reduces the overall execution time by dividing the work across multiple processors.

```
Processor 1:  A(1) = A(1) + S  Processor 2:  A(2) = A(2) + S  ...
```

3. Vector-Concurrent Mode: In vector-concurrent mode, the system utilizes both vector processors and concurrent scalar processors. While the vector processor works on vectorized operations for multiple elements, the scalar processors simultaneously handle different parts of the vector. This further reduces execution time by combining the advantages of vectorization and scalar parallelism.

```
Vector Processor:  A = A + S  Scalar Processors handle different subranges of A
```

4. Concurrent Outer/Vector Inner (COVI) Mode: In concurrent outer/vector inner (COVI) mode, the loop is split into two levels of parallelism. The outer loop is parallelized across multiple processors (each processor handling a portion of the loop), while the inner loop is vectorized. In this approach, each processor performs vector operations on different chunks of the array A, and vector processors execute operations on individual elements within each chunk concurrently.

```
Outer Loop:  Processor 1:  A(1:512) = A(1:512) + S  Processor 2:  A(513:1024) = A(513:1024)

          Vector Inner:  A = A + S   for each chunk of A
```

Conclusion: - In **Vector Mode**, the loop is vectorized to handle multiple elements in parallel, significantly speeding up execution. - In **Scalar-Concurrent Mode**, parallel scalar processors handle different loop iterations concurrently. - In **Vector-Concurrent Mode**, both vectorization and scalar parallelism are combined for further acceleration. - In **COVI Mode**, parallelism is introduced at both the outer loop level and inner loop level using vector processors, optimizing execution time by handling both data and tasks concurrently.

8.b)

Vectorizing the Code

Given the scalar loop:

```
Do I = 1, N A(I) = B(I) C(I) = A(I) + B(I) E(I) = C(I+1)Enddo
```

To vectorize this loop, the goal is to process multiple elements of the arrays A, B, C, and E simultaneously using vector processors. The following steps are involved in vectorizing the code:

1. Vectorizing the Assignment $A(I) = B(I)$: The entire array B can be copied to A in one vector operation. This eliminates the need for scalar iteration, as a vector processor can transfer multiple elements at once.

$$A = B$$

2. Vectorizing the Addition $C(I) = A(I) + B(I)$: This is an element-wise addition. The vector processor can add corresponding elements of vectors A and B simultaneously, producing the result vector C.

$$C = A + B$$

3. Vectorizing the Assignment $E(I) = C(I+1)$: Since $E(I) = C(I+1)$, a vector shift operation can be applied to vector C to produce E, effectively shifting the vector by one position.

$$E = \text{SHIFT}(C, 1)$$

This vector shift operation aligns $C(I + 1)$ with $E(I)$, handling all elements in parallel.

Summary of Vectorized Code:

$A = B$ (Copy B to A)$C = A+B$ (Element-wise addition)$E = \text{SHIFT}(C, 1)$ (Shift C to align $C(I + 1)$ wit

Conclusion: By using vector operations like element-wise addition and vector shifting, the entire loop is transformed into a series of vector operations, allowing the processor to handle multiple elements concurrently and significantly reducing execution time.

year question 2016

1.b)Daw the architecture of a vector super computer and discuss its major functional units.

Architecture of a Vector Supercomputer

A vector supercomputer is a high-performance system designed to efficiently handle large-scale scientific and engineering computations by processing vector data (arrays of values) in parallel. This architecture allows for significant speedup in computation time compared to scalar processors, particularly for tasks involving large datasets.

Major Functional Units:

1. **Vector Processor (VP):** The vector processor is the central component in a vector supercomputer. It is optimized for performing operations on vectors, where multiple elements of a vector are processed simultaneously. The vector processor consists of:

- Vector Registers: Store entire vectors for parallel processing.
- Functional Units: Execute arithmetic or logical operations on the vector elements.
- Vector Pipeline: Multiple stages of computation are overlapped to process vector elements efficiently.

2. **Scalar Processor (SP):** While the vector processor handles vector operations, the scalar processor is responsible for performing scalar arithmetic and control operations. It handles tasks like managing program flow, performing conditional checks, and managing input/output operations. It supports operations that are not vectorizable.

3. **Memory Unit:** The memory system in a vector supercomputer typically includes a combination of:

- Local Memory: Each vector processor has its own local memory, enabling fast access to data.
- Shared Memory: A shared memory system is used for global data access across multiple processors.

This hierarchical memory system minimizes bottlenecks and supports high-speed data access for vector processing.

4. **Interconnection Network:** The interconnection network facilitates data transfer between vector processors, scalar processors, and memory units. It ensures efficient communication and synchronization between processing units, crucial for maintaining high throughput in parallel operations.

5. **Control Unit:** The control unit coordinates the execution of instructions and manages the synchronization between the vector and scalar processors.

It ensures that vector and scalar operations are executed in harmony and handles the program's control flow.

6. **I/O Unit:** The I/O unit handles the data exchange between the supercomputer and external systems (e.g., storage, external devices). It ensures smooth data transfer to and from peripheral devices.

Conclusion: The architecture of a vector supercomputer is optimized for performing large-scale, parallel vector operations. By combining vector processors, scalar processors, hierarchical memory, high-speed interconnects, and specialized pipelines, vector supercomputers deliver exceptional performance for computationally intensive tasks. Each functional unit works together to handle different aspects of computation and communication, making vector supercomputers highly efficient for scientific research, simulations, and data processing.

1.c)Define implicit and explicit parallelism.

Implicit and Explicit Parallelism

Parallelism in computing refers to the ability to execute multiple operations simultaneously to improve performance. It can be classified into two types: implicit and explicit parallelism.

1. Implicit Parallelism: Implicit parallelism refers to the automatic detection and execution of parallel operations by the compiler or hardware without direct programmer intervention. The system identifies independent tasks and executes them concurrently.

Characteristics:

- The compiler or hardware manages parallelism automatically.
- No need for explicit parallel constructs in the program.
- Relies on techniques like instruction-level parallelism (ILP) and out-of-order execution.

Example: Modern superscalar processors execute multiple instructions in parallel using instruction pipelining.

2. Explicit Parallelism: Explicit parallelism requires the programmer to define parallelism manually using parallel constructs, such as multithreading, message passing, or parallel loops.

Characteristics:

- The programmer explicitly writes parallel code using APIs (e.g., OpenMP, MPI, CUDA).
- Requires manual synchronization and task distribution.
- More control over parallel execution but increases programming complexity.

Example: Using OpenMP in C to execute a loop in parallel:

```
#pragma omp parallel for
for(int i = 0; i < N; i++) {
    A[i] = B[i] + C[i];
}
```

Conclusion: Implicit parallelism is handled automatically by the system, while explicit parallelism requires programmer intervention. Both approaches aim to enhance performance but differ in complexity and control.

1.d)Derive MIPS rate and throughput rate.

MIPS Rate and Throughput Rate

1. MIPS Rate (Million Instructions Per Second): MIPS measures the instruction execution speed of a processor and is given by:

$$\text{MIPS} = \frac{\text{Clock Speed}}{\text{CPI} \times 10^6} \tag{3}$$

where:

- Clock Speed = Processor frequency in Hz
- CPI = Cycles Per Instruction

2. Throughput Rate: Throughput represents the number of instructions executed per second:

$$\text{Throughput} = \frac{\text{Total Instructions Executed}}{\text{Total Execution Time}} \tag{4}$$

For a pipelined system:

$$\text{Throughput} \approx \frac{\text{Clock Speed}}{\text{Pipeline Depth}} \tag{5}$$

Conclusion: MIPS rate determines instruction execution speed, while throughput measures overall performance based on completed tasks per unit time.

3.a)GIve the equation for pipeline CPI.How is ti related to ideal pipeline CPI?Explain the term involved it.

Pipeline CPI and Its Relation to Ideal Pipeline CPI

1. Equation for Pipeline CPI: The average **Cycles Per Instruction (CPI)** for a pipelined processor is given by:

$$\text{CPI}_{\text{pipeline}} = \text{CPI}_{\text{ideal}} + \frac{\text{Stall Cycles}}{\text{Total Instructions}} \tag{6}$$

2. Relation to Ideal Pipeline CPI: The **ideal pipeline CPI** is the best-case CPI when there are no stalls or hazards. In an n-stage pipeline, the ideal CPI is:

$$\text{CPI}_{\text{ideal}} = 1 \tag{7}$$

However, due to hazards (such as data, structural, and control hazards), stalls occur, increasing the effective CPI beyond the ideal value.

3. Explanation of Terms:

- $\text{CPI}_{\text{pipeline}}$ = Actual CPI in the presence of stalls.
- $\text{CPI}_{\text{ideal}}$ = Best-case CPI (usually 1 for an ideal pipeline).
- Stall Cycles = The number of extra clock cycles due to pipeline stalls.
- Total Instructions = The total number of instructions executed.

4. Conclusion: For an efficient pipeline, reducing stall cycles is critical to achieving CPI close to the ideal value. Techniques such as instruction reordering, forwarding, and branch prediction help minimize stalls and improve pipeline efficiency.

3.b)What are pipeline regu=isters,what sorts of information do they carry,and why are they needed?

Pipeline Registers and Their Importance

1. Definition: Pipeline registers are special storage elements placed between different stages of a pipeline to store intermediate data and control signals as instructions progress through the pipeline.

2. Information Carried: Each pipeline register holds:

- **Instruction Data:** The instruction opcode and operands.
- **Intermediate Results:** Computed values from previous pipeline stages.
- **Control Signals:** Signals required for instruction execution (e.g., ALU control, memory access).
- **Program Counter (PC):** Address of the instruction being processed.

3. Need for Pipeline Registers: Pipeline registers are essential because:

- They ensure data flows correctly between pipeline stages without loss.
- They allow independent execution of multiple instructions simultaneously, improving throughput.
- They help synchronize different stages and reduce hazards by maintaining proper data separation.

Conclusion: Pipeline registers enable efficient instruction execution by storing essential intermediate data, ensuring proper sequencing, and maintaining high performance.

3.c)Illustrate the operational principles to design of a pipeline floating-point adder with th inputs of A= a*2^p and$B = 2 * 2^q$

Pipeline Floating-Point Adder Design

1. Floating-Point Addition Steps: A floating-point adder follows these main steps:

1. **Exponent Alignment:** The operand with the smaller exponent is shifted right to match the larger exponent.
2. **Mantissa Addition:** The aligned mantissas are added.
3. **Normalization:** The result is adjusted to maintain proper floating-point representation.
4. **Rounding:** The final result is rounded to fit precision limits.

2. Pipeline Stages for Floating-Point Addition: To improve performance, these steps are divided into pipeline stages:

- **Stage 1 (Exponent Comparison and Alignment)** - Compare exponents p and q. - Shift the mantissa of the smaller exponent.
- **Stage 2 (Mantissa Addition)** - Perform addition of aligned mantissas:

$$M_{\text{sum}} = M_A + M_B$$

- **Stage 3 (Normalization)** - Adjust mantissa and update exponent to keep proper form.
- **Stage 4 (Rounding and Output)** - Apply rounding rules and store the final result.

3. Example with Inputs $A = a \times 2^p$ and $B = 2 \times 2^q$:

- If $p > q$, shift B's mantissa (2) right by $p - q$ places.
- Add mantissas.
- Normalize and round the result.

4. Conclusion: Pipelining the floating-point adder ensures higher throughput by allowing multiple additions to be processed in different stages simultaneously.

4.a)Define network diameter,bisection width and degree of parallelism.

Definitions: Network Diameter, Bisection Width, and Degree of Parallelism

1. Network Diameter (D): Network diameter is the longest shortest path between any two nodes in a network. It represents the maximum communication delay in terms of the number of hops.

$$D = \max_{i,j} d(i,j) \tag{8}$$

where $d(i,j)$ is the shortest distance (in hops) between nodes i and j.

2. Bisection Width (B): Bisection width is the minimum number of links that must be removed to divide the network into two equal halves. It measures the communication bandwidth of the network.

3. Degree of Parallelism (P): Degree of parallelism refers to the maximum number of independent operations that can be executed simultaneously in a parallel system. It is often related to the number of processors or processing units available.

Conclusion:

- A smaller network diameter ensures lower communication latency.
- A higher bisection width improves data transfer capability.
- A greater degree of parallelism leads to better computational efficiency.

4.b)How to compute average parallelism.

Average Parallelism

Definition: Average parallelism (P_{avg}) measures the level of concurrency in a parallel system. It is defined as:

$$P_{\text{avg}} = \frac{\sum_{i=1}^{N} T_i \times P_i}{\sum_{i=1}^{N} T_i} \tag{9}$$

where:

- P_i = Parallelism level (number of processors used) in phase i.
- T_i = Execution time spent in phase i.
- N = Number of execution phases.

Interpretation: - A higher P_{avg} indicates better utilization of parallel resources. - If $P_{\text{avg}} = 1$, the execution is purely sequential. - If P_{avg} is close to the maximum available processors, the system achieves high parallel efficiency.

Conclusion: Average parallelism helps evaluate how well a system exploits multiple processing units over time.

4.c)Ruby Lee has defined several parameter for calculating parallel computations.Derive the parameters.

Ruby Lee's Parameters for Parallel Computation

Ruby Lee defined key parameters to evaluate parallel computations:

1. Speedup (S)

$$S = \frac{T_s}{T_p} \tag{10}$$

where:

- T_s = Execution time in a sequential system.
- T_p = Execution time in a parallel system.

2. Efficiency (E)

$$E = \frac{S}{P} = \frac{T_s}{P \times T_p} \tag{11}$$

where:

- P = Number of processors.

Efficiency shows how well processors are utilized.

3. Amdahl's Law for Parallelization Fraction (f)

$$S = \frac{1}{(1-f) + \frac{f}{P}} \tag{12}$$

where:

- f = Fraction of the program that can be parallelized.

Conclusion: These parameters help measure the effectiveness of parallel computation in terms of speed, efficiency, and workload distribution.

5.a)Explain in a VLIW architecture.

VLIW (Very Long Instruction Word) Architecture

Definition: VLIW is a type of processor architecture where multiple operations are encoded in a single long instruction word. This allows the processor to execute several operations in parallel during one clock cycle.

Key Characteristics:

- **Multiple Operations in One Instruction:** Each instruction word contains multiple independent operations, which can be executed in parallel.

- **Compiler-Controlled Parallelism:** The compiler is responsible for scheduling instructions to maximize parallelism, rather than the hardware.
- **Fixed Instruction Length:** Each instruction is typically of a fixed size, containing several sub-instructions for different functional units.

Advantages:

- Efficient use of hardware resources.
- High throughput and performance for well-optimized code.

Conclusion: VLIW architecture improves parallelism by issuing multiple operations simultaneously, relying on the compiler to optimize instruction scheduling.

5.b)Compare the PRAM and VLSI models of parallel computers and mention how these models facilitate the study of asympototic behavior of algorithms implementation on parallel computers.

Comparison between PRAM and VLSI Models of Parallel Computers

1. PRAM (Parallel Random Access Machine) Model:

The PRAM model is an abstract theoretical model used to study parallel algorithms. It assumes the existence of multiple processors with access to a shared memory, and these processors can perform operations in parallel with simultaneous memory access.

- **Assumptions:**
  - Unlimited processors.
  - Instantaneous communication between processors.
  - No memory contention (simultaneous access to memory by different processors).
- **Focus:** This model is used to analyze the asymptotic time complexity of parallel algorithms in an idealized environment.

2. VLSI (Very Large Scale Integration) Model:

The VLSI model is based on the physical hardware implementation of parallel systems. It takes into account the actual constraints of the hardware, such as processor interconnections, memory hierarchy, and communication delays.

- **Assumptions:**
  - Limited number of processors.

  - Communication delays and physical wiring constraints.
  - Memory is divided and distributed (distributed memory systems).

- **Focus:** This model helps study the realistic performance of parallel algorithms, taking hardware-specific factors into consideration.

Comparison:

- **Idealized vs. Realistic:** The PRAM model is idealized and assumes infinite resources and no communication delays, whereas the VLSI model is hardware-based, considering the limitations of real systems.
- **Algorithm Analysis:** The PRAM model is used to study the theoretical asymptotic behavior of parallel algorithms, often without worrying about real-world hardware constraints. The VLSI model, however, allows researchers to study the actual behavior of parallel algorithms considering communication and computation costs in real hardware.

Conclusion:
Both models facilitate the study of parallel algorithms but in different contexts. The PRAM model is used for theoretical analysis, while the VLSI model helps understand the practical implementation and behavior of algorithms on real hardware, considering both computation and communication costs.

5.c)Differentiate the three different shared memory system classification:UMA,NUMA,and COMA

Comparison of UMA, NUMA, and COMA Shared Memory Systems

1. UMA (Uniform Memory Access):

- In UMA, all processors have equal access time to the system's shared memory.
- **Memory Access:** Memory is shared uniformly, and each processor can access any part of memory in the same amount of time.
- **System Type:** Typically used in symmetric multiprocessor (SMP) systems.
- **Performance:** Ideal for small to medium-scale systems where memory access latency is not a concern.

2. NUMA (Non-Uniform Memory Access):

- In NUMA, memory access time depends on the memory location relative to the processor.
- **Memory Access:** Processors can access their local memory faster than remote memory. The access time is non-uniform.

- **System Type:** Common in large multiprocessor systems and is often used in systems where scalability is important.
- **Performance:** Performance is optimized by placing data close to the processors that will access it most frequently.

3. COMA (Cache-Only Memory Architecture):

- In COMA, memory is organized as a large cache that is distributed across processors.
- **Memory Access:** Memory is not shared directly; instead, each processor has its own cache, and the caches are used for data storage and communication.
- **System Type:** Often used in distributed memory systems where processors communicate via message passing.
- **Performance:** Helps reduce memory bottlenecks by utilizing the cache for fast access, but requires efficient cache management and coherence protocols.

Comparison Summary:

- **UMA:** Equal memory access time for all processors; best for small systems.
- **NUMA:** Memory access time depends on the processor's distance from the memory; scalable for large systems.
- **COMA:** Memory is managed as caches distributed across processors, providing high-performance access but complex cache management.

Conclusion: UMA is ideal for systems with fewer processors, NUMA is better for large-scale systems with many processors, and COMA focuses on reducing memory access latency through efficient cache management.

6.a)Define distributed system.Why distributed systems to be needed.

Distributed System:

A **distributed system** is a collection of independent computers that appear to the user as a single coherent system. These computers communicate and coordinate their actions by passing messages over a network.

Why Distributed Systems are Needed:

- **Resource Sharing:** Distributed systems allow the sharing of resources such as hardware, software, and data across multiple machines.

- **Scalability:** They can scale easily by adding more machines to handle increased load or to provide more services.
- **Reliability:** Distributing tasks across multiple machines improves fault tolerance. If one machine fails, others can take over the workload.
- **Cost-Effectiveness:** It allows the use of inexpensive, off-the-shelf hardware while still providing powerful computational capabilities.

Conclusion: Distributed systems are essential for resource sharing, scalability, reliability, and cost-effective computation in modern computing environments.

6.b)Differentiate between parallel and distributd systems

Difference Between Parallel and Distributed Systems

Parallel Systems	**Distributed Systems**
Consist of multiple processors that work on a single problem simultaneously.	Consist of multiple independent computers that work together over a network.
Processors share memory and have tight coupling.	Nodes have independent memory and communicate via message passing.
Aimed at achieving high-speed computation through parallel execution of tasks.	Focuses on resource sharing, fault tolerance, and scalability.
Requires specialized hardware such as multi-core or multi-processor systems.	Uses general-purpose computers connected through a network.
Low communication overhead due to shared memory.	Higher communication overhead due to network-based interactions.
Example: Supercomputers like IBM Blue Gene performing scientific simulations.	Example: Cloud computing services like Google Drive or Amazon Web Services.

Conclusion: Parallel systems focus on executing tasks simultaneously using shared or tightly coupled memory, while distributed systems emphasize independent machines communicating over a network, providing greater scalability and fault tolerance.

6.c)What are the common charecteristics that can be used to access distributed systems.

Common Characteristics to Access Distributed Systems:

- **Transparency:** The system should hide the complexities of the distributed environment, such as location, access, and replication transparency.
- **Scalability:** The system must be able to scale efficiently by adding more resources or nodes without significant performance degradation.
- **Fault Tolerance:** Distributed systems should handle failures gracefully and continue to operate even if some components fail.
- **Concurrency:** Multiple processes or users should be able to access the system simultaneously without conflict.
- **Security:** Security mechanisms must be in place to ensure data integrity, privacy, and authorized access across distributed nodes.

Conclusion: These characteristics help in efficiently accessing and utilizing distributed systems, ensuring robustness and performance in a distributed environment.

6.d)What are the advantages and disadvantages of distributed system?

Advantages and Disadvantages of Distributed Systems

Advantages:

- **Scalability:** Distributed systems can easily scale by adding more nodes or resources to the system.
- **Fault Tolerance:** They are designed to handle failures gracefully, allowing the system to continue functioning even if some components fail.
- **Resource Sharing:** Distributed systems allow sharing of resources (hardware, software, and data) across different machines, improving efficiency.
- **Cost Efficiency:** Distributed systems can use inexpensive, off-the-shelf components, making them more cost-effective than centralized systems.
- **Flexibility:** They allow diverse types of machines and operating systems to be part of the network.

Disadvantages:

- **Complexity:** Designing and managing a distributed system is more complex due to factors like network communication, data consistency, and fault tolerance.
- **Latency:** Network communication between distributed nodes can introduce delays, which can affect system performance.
- **Security Challenges:** Ensuring security in a distributed system is more difficult due to the larger number of entry points and data transfer across networks.
- **Data Consistency:** Maintaining data consistency across distributed nodes is challenging, especially in systems with frequent updates.
- **Dependency on Network:** Distributed systems heavily rely on the network infrastructure, and poor network performance can severely impact system operations.

Conclusion: While distributed systems offer scalability, fault tolerance, and resource sharing, they come with challenges in terms of complexity, security, latency, and data consistency.

7.b)Show how to reduce minimal average latency by inserting monocompute delay.

Reducing Minimal Average Latency by Inserting Monocompute Delay

Concept: In many parallel systems, latency is a critical performance factor, and reducing it can greatly improve system throughput. The minimal average latency can be reduced by introducing **monocompute delay**, which involves adding a fixed amount of delay or synchronization to the computation process to achieve better load balancing and coordination among tasks.

Steps to Insert Monocompute Delay:

1. **Identify Critical Sections:** - Analyze the parallel computation to identify critical sections or tasks that are causing delays due to unbalanced workload distribution or synchronization issues.

2. **Introduce Delays in Parallel Tasks:** - Insert a small, controlled monocompute delay in tasks that have faster execution times to align with the slower tasks. This helps balance the execution times of different parallel tasks, thus reducing the overall time spent waiting for faster tasks.

3. **Optimization:** - The inserted delay should be carefully tuned to ensure that it does not lead to unnecessary idling of resources. It should only be enough to prevent faster tasks from completing too early and waiting for slower tasks.

4. **Evaluate System Performance:** - After inserting the delay, measure the average latency and adjust the delay duration accordingly. The goal is to achieve a minimal average latency by ensuring that all parallel tasks are well-coordinated without overloading any single resource.

Example:

Consider a parallel system with two tasks: Task A (fast) and Task B (slow). Without any delay, Task A finishes early and has to wait for Task B to finish. By inserting a monocompute delay in Task A, you can make both tasks complete at similar times, reducing the idle time and improving the minimal average latency of the system.

Conclusion: Inserting monocompute delay helps to reduce minimal average latency by synchronizing task completion times, optimizing resource utilization, and improving system throughput in parallel computing environments.

8.a)Explain how improve instuction scheduling in the CDC 6600 processor.

Improving Instruction Scheduling in the CDC 6600 Processor

The CDC 6600, one of the first supercomputers, introduced several innovative techniques for improving instruction scheduling, enabling high-performance execution. These improvements are crucial for maximizing the efficiency of the CPU, especially in parallel processing environments.

Key Strategies for Improving Instruction Scheduling:

1. **Multiple Functional Units:** - The CDC 6600 features multiple functional units that can execute different types of instructions in parallel. For example, there are separate units for integer arithmetic, floating-point operations, and memory access. Instruction scheduling can be optimized by ensuring that instructions are dispatched to the appropriate functional units, minimizing idle times and avoiding pipeline stalls.

2. **Overlapping Instruction Execution:** - The CDC 6600 can execute instructions in an overlapping fashion. Instruction scheduling is done in such a way that independent instructions are executed simultaneously across multiple functional units, rather than sequentially. This helps in maximizing throughput and reducing execution time by utilizing the parallelism inherent in the processor architecture.

3. **Asynchronous Execution and Register Renaming:** - To handle dependencies and hazards between instructions, the CDC 6600 uses asynchronous execution. It allows instructions to be scheduled independently as long as there are no data dependencies between them. Additionally, the use of register renaming helps eliminate false dependencies (write-after-write and read-after-write) by dynamically assigning physical registers to instructions.

4. **Compiler Support for Instruction Scheduling:** - The CDC 6600 relies on advanced compilers that can analyze dependencies and reorder instructions for optimal execution. The compiler rearranges instructions to minimize stalls and maximize the use of available functional units. The scheduling takes into account both the data dependencies and the availability of resources to reduce bottlenecks in the execution pipeline.

Example:

Consider a scenario where the execution of an arithmetic operation and a memory load operation are independent of each other. In a naive scheduling approach, these might be executed sequentially. However, in the CDC 6600, these operations can be scheduled in parallel, each going to different functional units, thus speeding up the overall execution time.

Conclusion: The CDC 6600 improves instruction scheduling by utilizing multiple functional units, overlapping execution, asynchronous operations, register renaming, and compiler optimizations, ensuring that resources are efficiently utilized and execution time is minimized.

8.b)Diss different types of vector instructions.

Different Types of Vector Instructions

Vector instructions are designed to perform operations on entire vectors (arrays of data) in parallel. These instructions allow efficient computation by processing multiple data elements in a single instruction cycle. There are several types of vector instructions, each optimized for specific tasks.

1. Vector Load and Store Instructions:

- **Definition:** These instructions are used to transfer data between memory and vector registers.
- **Characteristics:**
  - **Vector Load**: Loads a vector from memory into a vector register.
  - **Vector Store**: Stores the contents of a vector register into memory.
- **Example:**
  - `VLOAD VectorRegister, MemoryAddress` (loads a vector from memory).
  - `VSTORE MemoryAddress, VectorRegister` (stores a vector to memory).

2. Vector Arithmetic Instructions:

- **Definition:** These instructions perform arithmetic operations on vectors.
- **Characteristics:**
  - These instructions apply operations such as addition, subtraction, multiplication, etc., element-wise to each component of a vector.
  - They execute in parallel on all elements of the vector.
- **Example:**
  - `VADD VectorA, VectorB, VectorC` (adds corresponding elements of VectorA and VectorB, storing the result in VectorC).

- `VMUL VectorA, VectorB, VectorC` (multiplies corresponding elements of VectorA and VectorB).

3. Vector Reduction Instructions:

- **Definition:** These instructions reduce a vector to a single scalar value by performing an operation across all elements of the vector.
- **Characteristics:**
  - Common operations include summing all elements of a vector, finding the maximum or minimum value, or computing a dot product.
  - The result is a single scalar value derived from the vector.
- **Example:**
  - `VREDUCEADD VectorA, ScalarResult` (sums all elements of VectorA).
  - `VREDUCEMAX VectorA, ScalarResult` (finds the maximum value in VectorA).

4. Vector Scatter and Gather Instructions:

- **Definition:** These instructions allow accessing non-contiguous elements of a vector or array.
- **Characteristics:**
  - **Scatter**: Writes elements of a vector to non-contiguous memory locations.
  - **Gather**: Reads elements from non-contiguous memory locations into a vector register.
- **Example:**
  - `VSCATTER VectorA, Indices, Memory` (scatters the elements of VectorA into non-contiguous memory locations based on Indices).
  - `VGATHER Memory, Indices, VectorB` (gathers data from non-contiguous memory locations into VectorB based on Indices).

Conclusion: Vector instructions, including load/store, arithmetic, reduction, and scatter/gather instructions, allow parallel processing of data and enhance performance in vector processors. These instructions enable efficient data manipulation, making them crucial for high-performance computing tasks.

8.c)Discuss s-access memory organization,c-access memory organization and c/s access memory organization.

S-access Memory Organization, C-access Memory Organization, and C/S Access Memory Organization

In modern computer systems, different memory organizations are used to optimize access patterns and improve performance. These organizations cater to varying needs in terms of memory access, latency, and synchronization between processors.

1. S-access Memory Organization:

- **Definition:** S-access (Single Access) memory organization involves memory that is accessed by a single processor or controller at any given time.
- **Characteristics:**
  - It is often used in systems where only one processor needs to access the memory.
  - There is no parallel access to memory from multiple processors, which simplifies the memory access mechanism.
  - Suitable for single-user or single-processor systems.
- **Example:** A simple personal computer or embedded system using a single processor to manage memory.

2. C-access Memory Organization:

- **Definition:** C-access (Concurrent Access) memory organization allows multiple processors or units to access the memory concurrently.
- **Characteristics:**
  - This type of memory organization supports parallel processing environments, where multiple processors work on the same data.
  - Ensures that data can be accessed simultaneously by multiple units, improving performance and throughput.
  - However, it requires sophisticated synchronization mechanisms to handle conflicts and ensure consistency.
- **Example:** Multi-core processors where all cores have access to shared memory.

3. C/S Access Memory Organization:

- **Definition:** C/S access (Client/Server) memory organization involves a system where memory access is divided between clients and servers.

- **Characteristics:**
  - In this system, the server manages the memory, and the client requests access to the data stored in the server's memory.
  - The server handles memory allocation, retrieval, and synchronization, while the clients access data based on the server's response.
  - This organization is commonly found in distributed systems where clients need access to shared resources.
- **Example:** A client-server database system where the server manages the database, and multiple client machines query and retrieve data.

Conclusion: Each memory organization has its advantages and is suited to different types of systems. S-access is simpler and used in single-processor systems, C-access is designed for parallel processing environments, and C/S access is ideal for distributed systems involving client-server interactions.

Differentiate betwenn centralized and disrtubuted database system

Centralized Database System	Distributed Database System
The entire database is stored at a single location (server).	The database is distributed across multiple sites connected via a network.
Single point of failure; if the central server crashes, the entire system becomes unavailable.	More fault-tolerant; failure of one site does not affect the entire system.
Easier to manage and maintain as all data is stored in one place.	Complex to manage due to data distribution and synchronization.
Low network overhead as all data processing occurs in one location.	High network overhead due to data transfer between multiple sites.
Limited scalability; performance bottlenecks can occur as user load increases.	Highly scalable; additional sites can be added to distribute the load.
Example: Traditional relational databases like MySQL running on a single server.	Example: Google Cloud Spanner, Apache Cassandra, Amazon DynamoDB.

What are the technical reasons for using hiearchical distributed processing?

Hierarchical distributed processing organizes computing resources into multiple levels, where higher levels manage lower levels. The key technical reasons for using this approach include:

- **Scalability:** Hierarchical structures efficiently manage large systems by distributing processing tasks across multiple levels, reducing bottlenecks.
- **Fault Tolerance:** Failures at lower levels do not affect the entire system, ensuring higher reliability.
- **Load Balancing:** Workloads can be dynamically distributed among different levels to optimize performance.
- **Reduced Communication Overhead:** Localized processing minimizes the need for global communication, improving efficiency.
- **Modularity and Manageability:** Easier to maintain, update, and troubleshoot as components are structured in a hierarchical manner.

- **Security and Access Control:** Different levels can enforce access policies, restricting unauthorized access to sensitive data.
- **Efficient Resource Utilization:** Processing tasks are delegated to appropriate levels, ensuring that computational resources are used optimally.
- **Support for Heterogeneous Systems:** Hierarchical structures allow integration of different hardware and software components across multiple levels.

Data should be centralized and decentralized.Explain.

Centralized Data: All data is stored in a single location. It ensures **consistency, security, and simplified management** but has **scalability issues and high latency**.

Decentralized Data: Data is distributed across multiple locations. It provides **fault tolerance, scalability, and faster access** but faces **consistency challenges and higher maintenance costs**.

Hybrid Approach: Combines both by using **centralized control** for **security and consistency** while distributing data for **scalability and performance**.